

CPU Scheduling

Luca Spalazzi

spalazzi@dii.univpm.it

www.univpm.it/luca.spalazzi



Dipartimento di Ingegneria
dell'Informazione



cini
Cyber Security Lab@UnivPM

Gruppo Ingegneria Informatica
@UnivPM





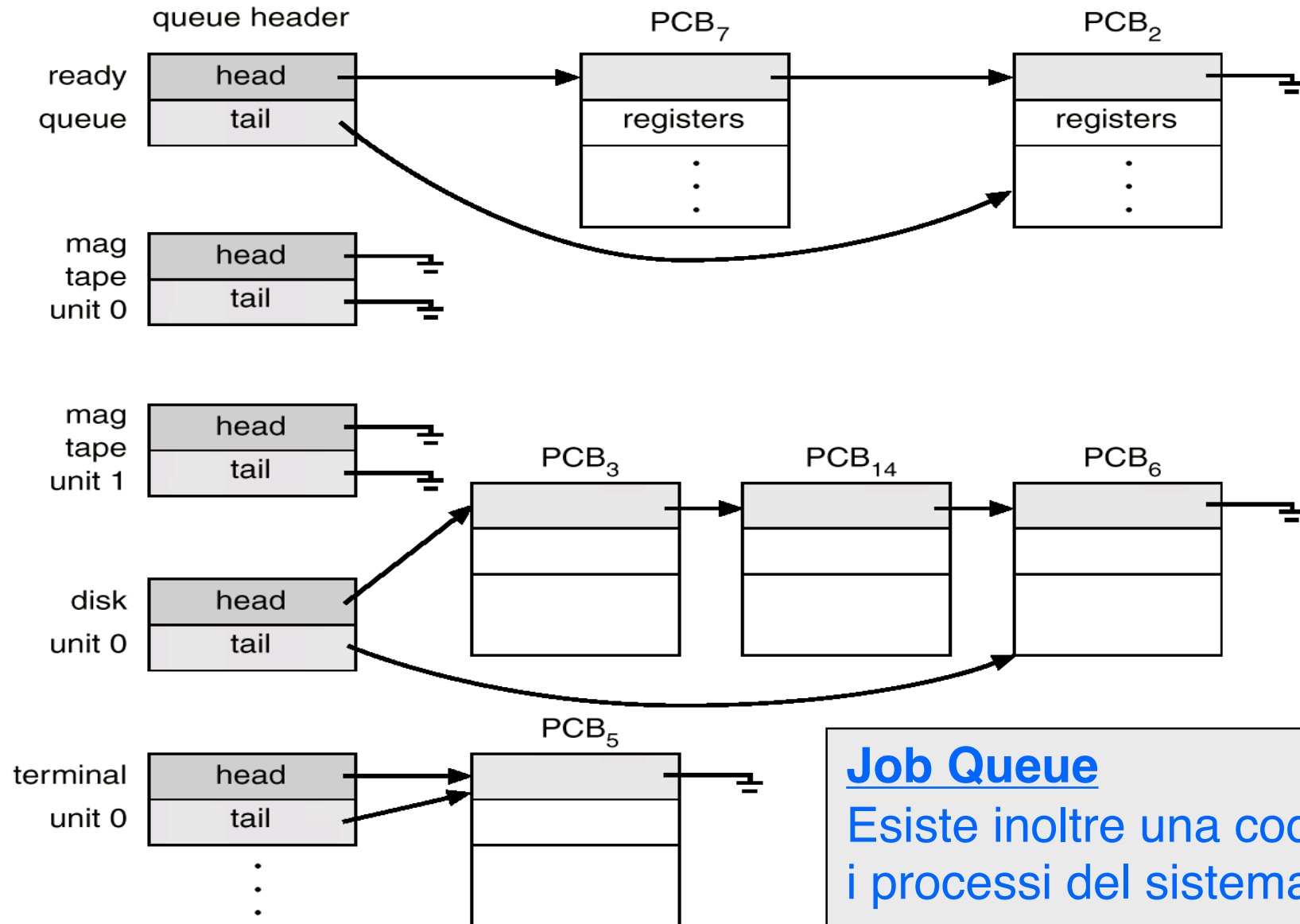
-
- Scheduling dei processi
 - CPU Scheduling nei sistemi monoprocesso
 - CPU Scheduling nei sistemi multiprocesso
 - Lo scheduling in Linux



CPU Scheduling

SCHEDULING DEI PROCESSI

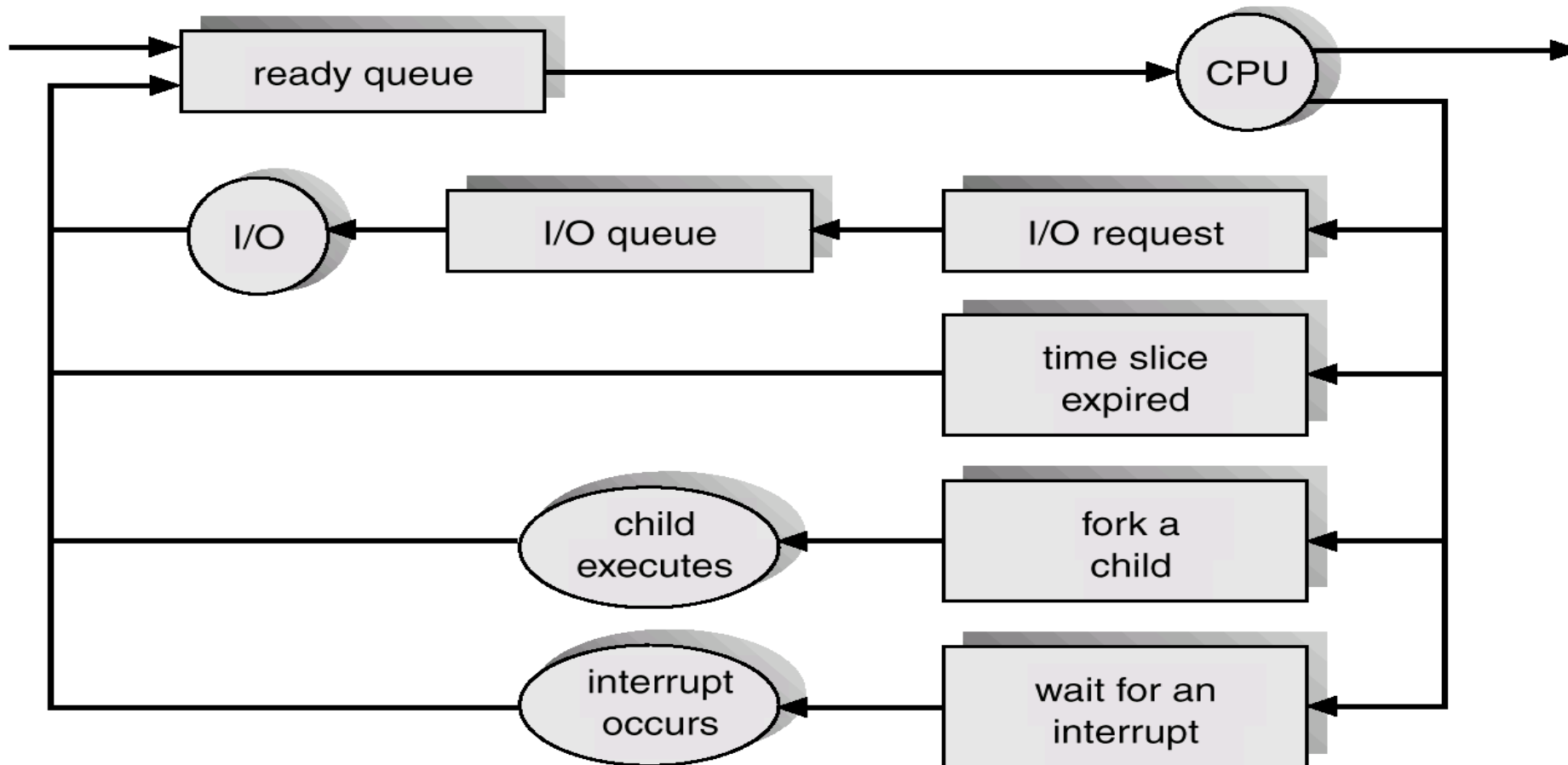
Code di Scheduling dei Processi



Job Queue

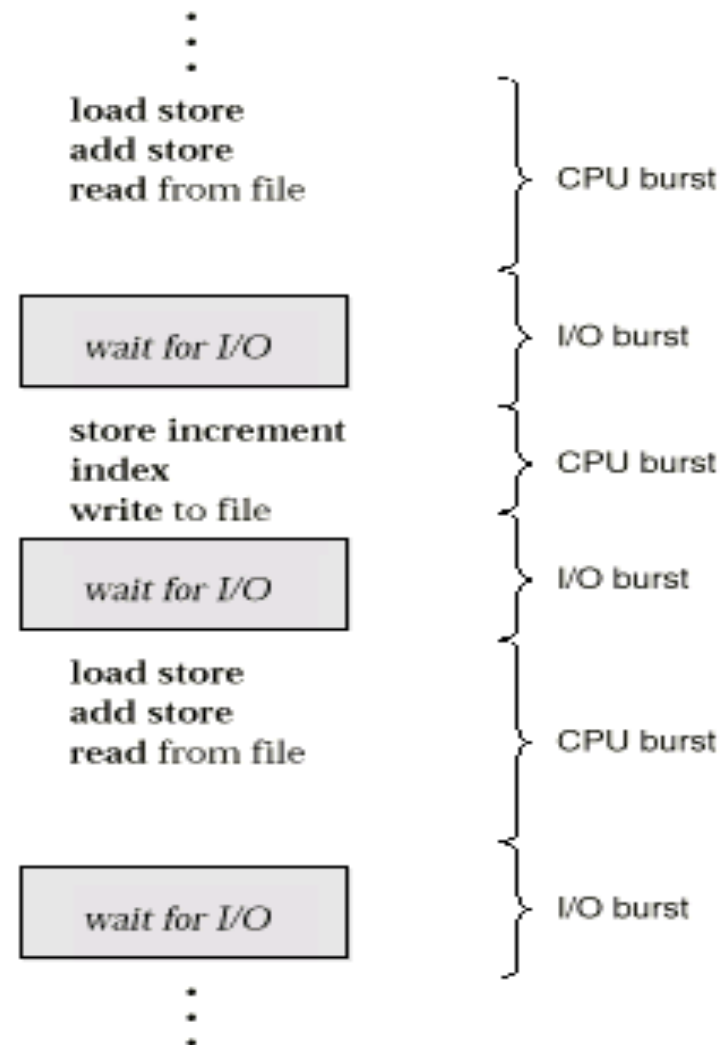
Esiste inoltre una coda di tutti i processi del sistema

Scheduling dei Processi

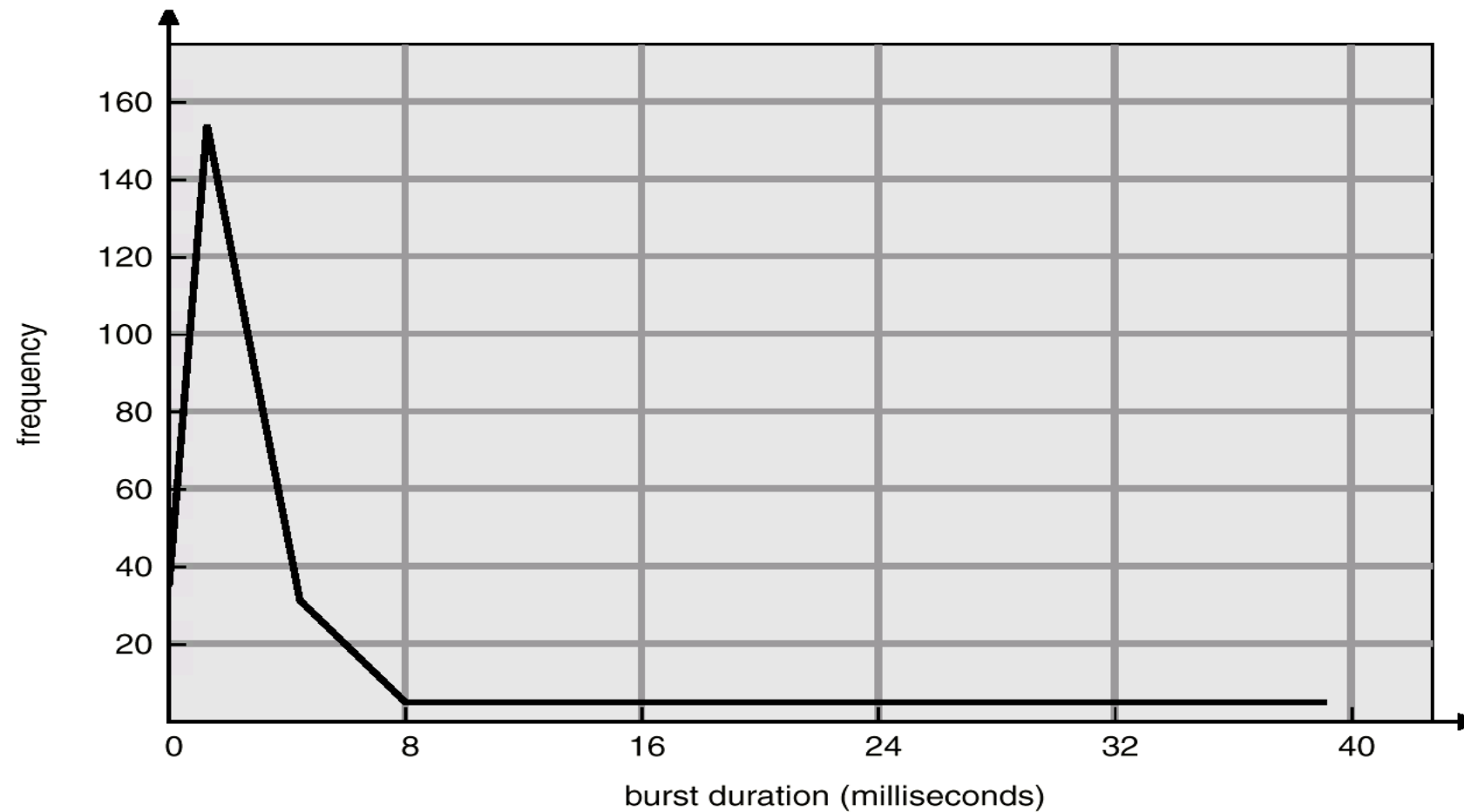


Sequenza di CPU Burst e I/O Burst

- L'esecuzione di un processo consiste di un ciclo di esecuzione nella CPU e di attesa dell'I/O.



Distribuzione dei CPU-burst





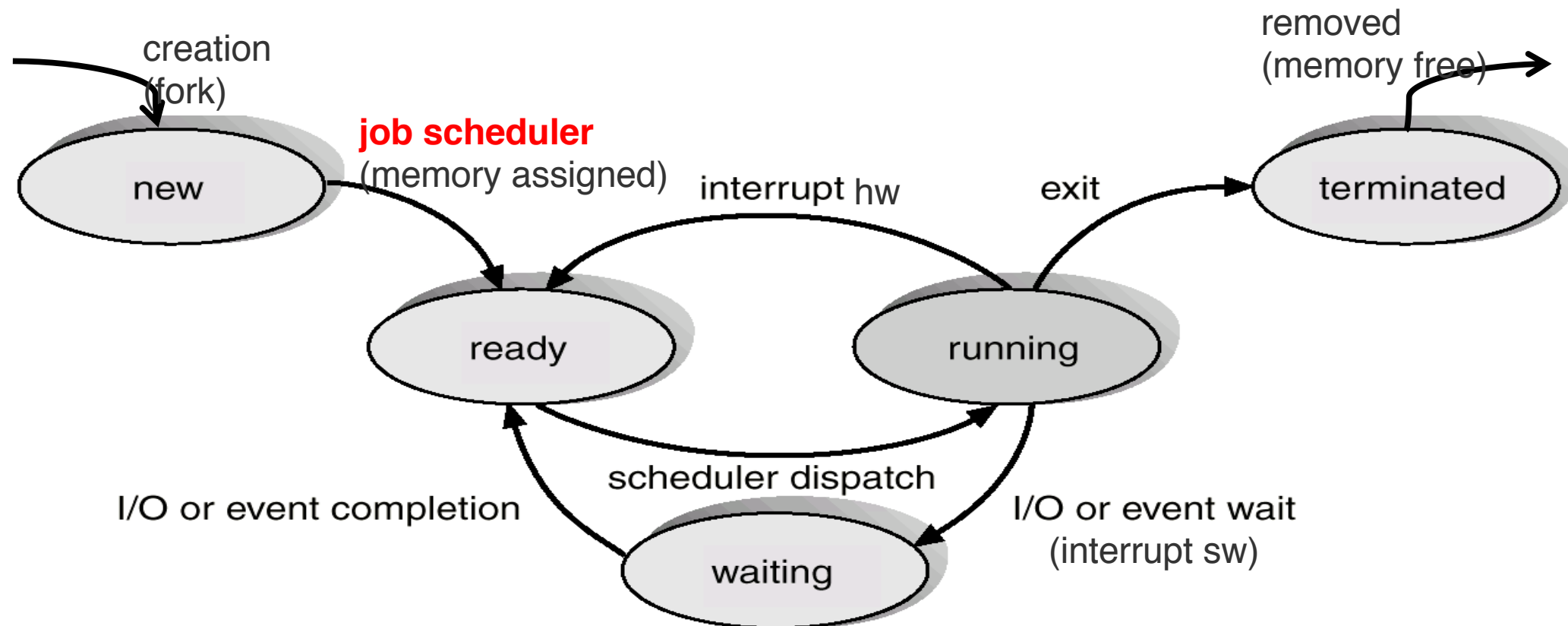
CPU-bound e I/O-bound

- I processi possono essere:
 - Processi I/O-bound – impiegano più tempo per l'I/O che per le elaborazioni, hanno molti CPU bursts di breve durata.
 - Processi CPU-bound – impiegano più tempo nelle elaborazioni che per l'I/O, hanno pochi CPU bursts ma di lunga durata.

- **Scheduler a lungo termine** (o job scheduler) – seleziona tra i processi in stato di new quelli da caricare in memoria.
 - E' invocato di rado (secondi, minuti) $\Rightarrow$ (può essere lento).
 - Deve massimizzare il grado di multiprogrammazione senza far degradare le prestazioni del sistema.
- **Scheduler a medio termine** – seleziona tra i processi “scaricati” dalla memoria (swapped-out) quelli da caricare (swap in) nuovamente in memoria.
 - E' invocato di rado (secondi) $\Rightarrow$ (può essere lento).
 - Deve evitare il degrado delle prestazioni del sistema.
- **Scheduler a breve termine** (o CPU scheduler) – seleziona tra i processi presenti nella ready queue il processo a cui allocare la CPU.
 - E' invocato di frequente (millisecondi) $\Rightarrow$ deve essere molto veloce.
 - Deve massimizzare l' utilizzo della CPU

Job Scheduler

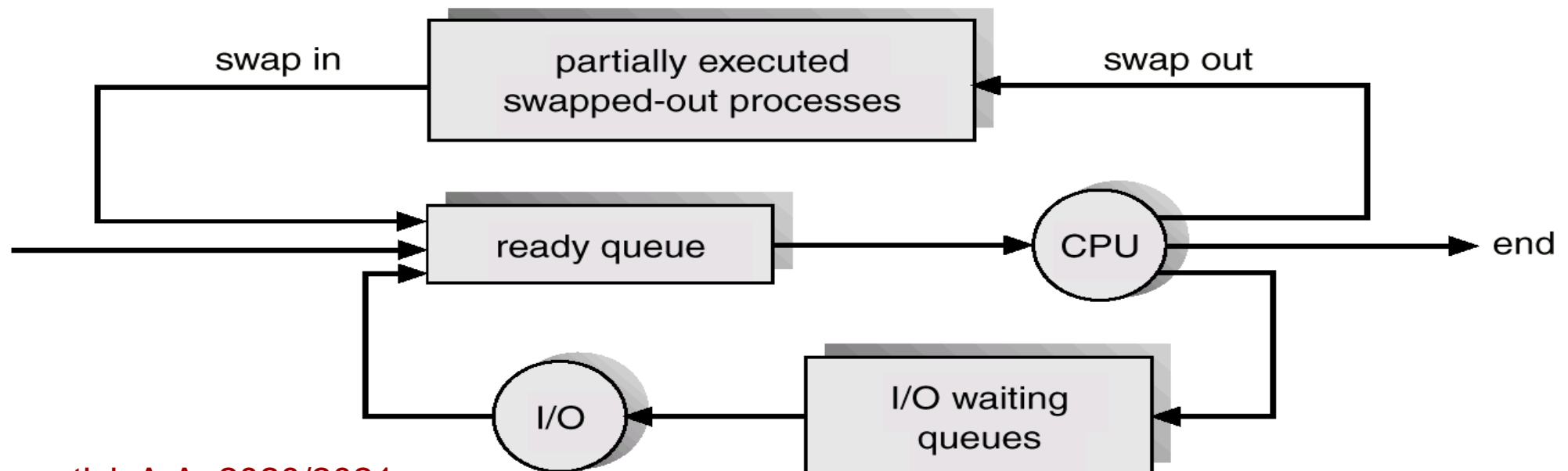
- Controlla il grado di multiprogrammazione (possono essere caricati nuovi processi in memoria senza provocare il trashing?).
- Se sì, seleziona i processi che devono essere caricati in memoria.
- I descrittori di tali processi vengono inseriti nella ready queue.
- Lo stato di tali processi passa da new a ready.



Middle-term Scheduler

Controlla il grado di multiprogrammazione ed il rischio di degrado delle prestazioni del sistema (trashing).

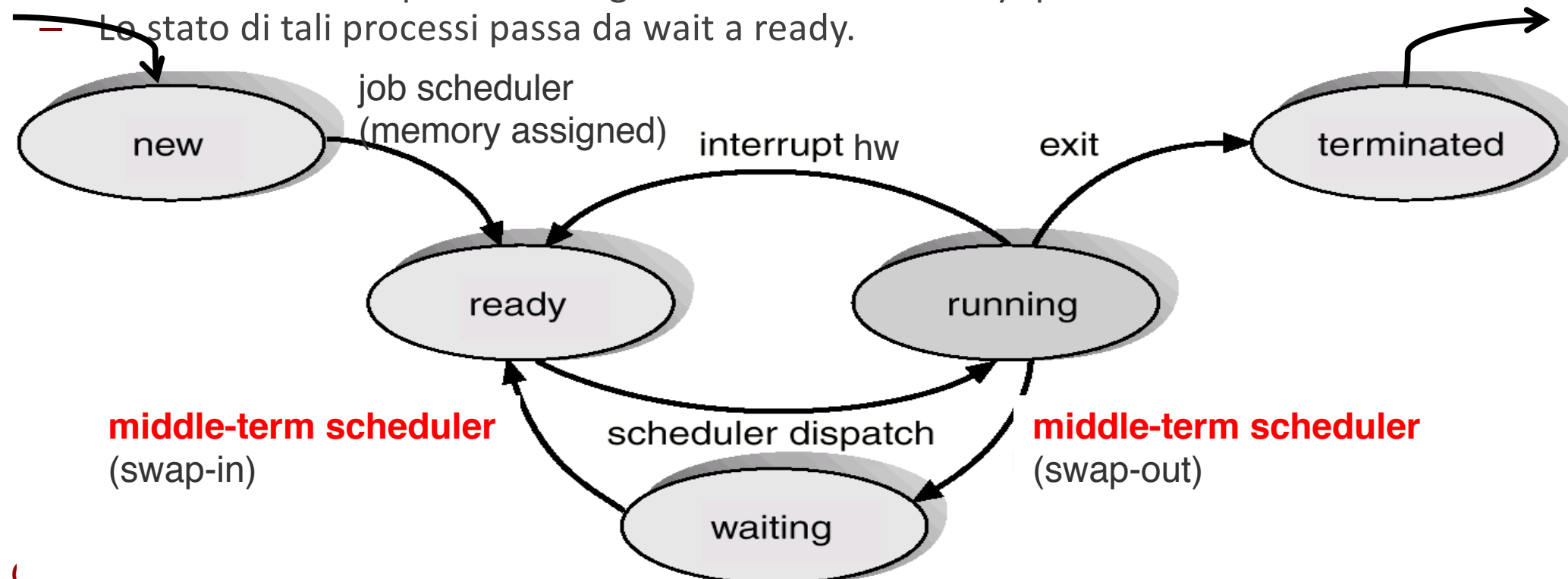
- Se il grado di multiprogrammazione è troppo elevato (il sistema rischia il trashing), seleziona i processi da scaricare (swap out).
 - Lo stato di tali processi passa a wait.
- Se il grado di multiprogrammazione è sufficientemente basso, seleziona i processi swapped-out che devono essere caricati in memoria (swap in).
 - I descrittori di tali processi vengono inseriti nella ready queue.
 - Lo stato di tali processi passa da wait a ready.



Middle-term Scheduler

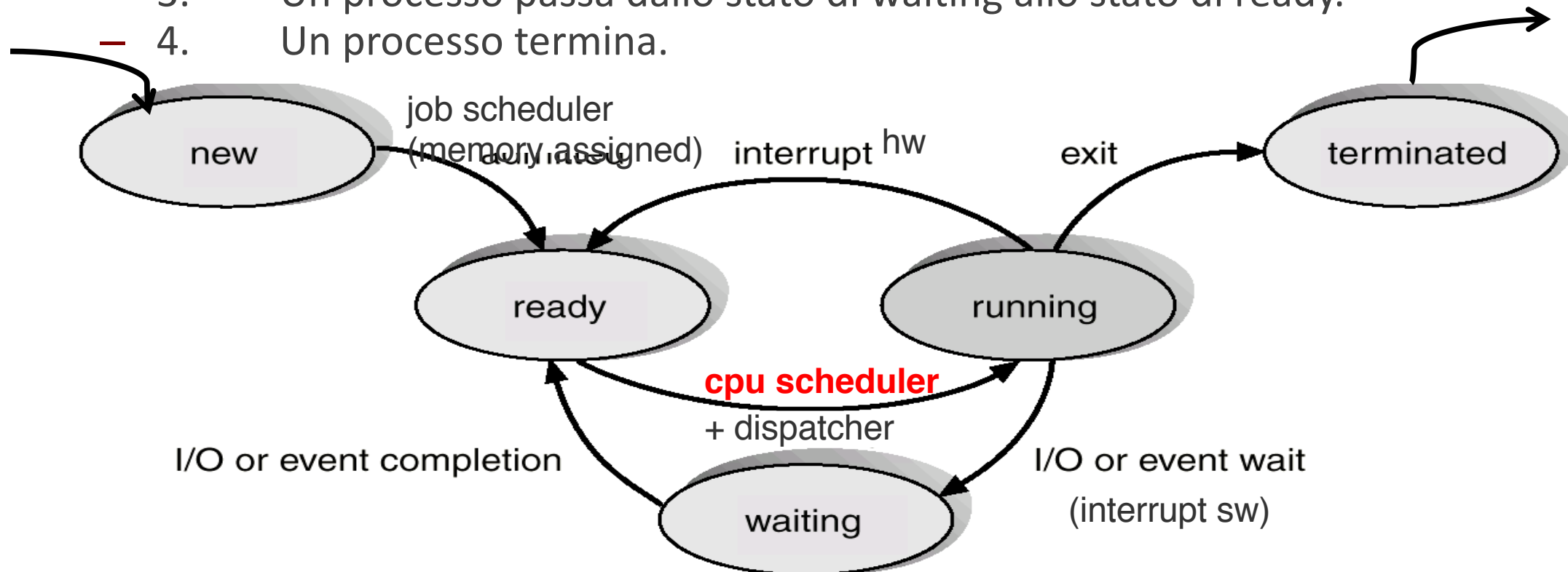
Controlla il grado di multiprogrammazione ed il rischio di degrado delle prestazioni del sistema (trashing).

- Se il grado di multiprogrammazione è troppo elevato (il sistema rischia il trashing), seleziona i processi da scaricare (swap out).
 - Lo stato di tali processi passa a wait.
- Se il grado di multiprogrammazione è sufficientemente basso, seleziona i processi swapped-out che devono essere caricati in memoria (swap in).
 - I descrittori di tali processi vengono inseriti nella ready queue.
 - Lo stato di tali processi passa da wait a ready.



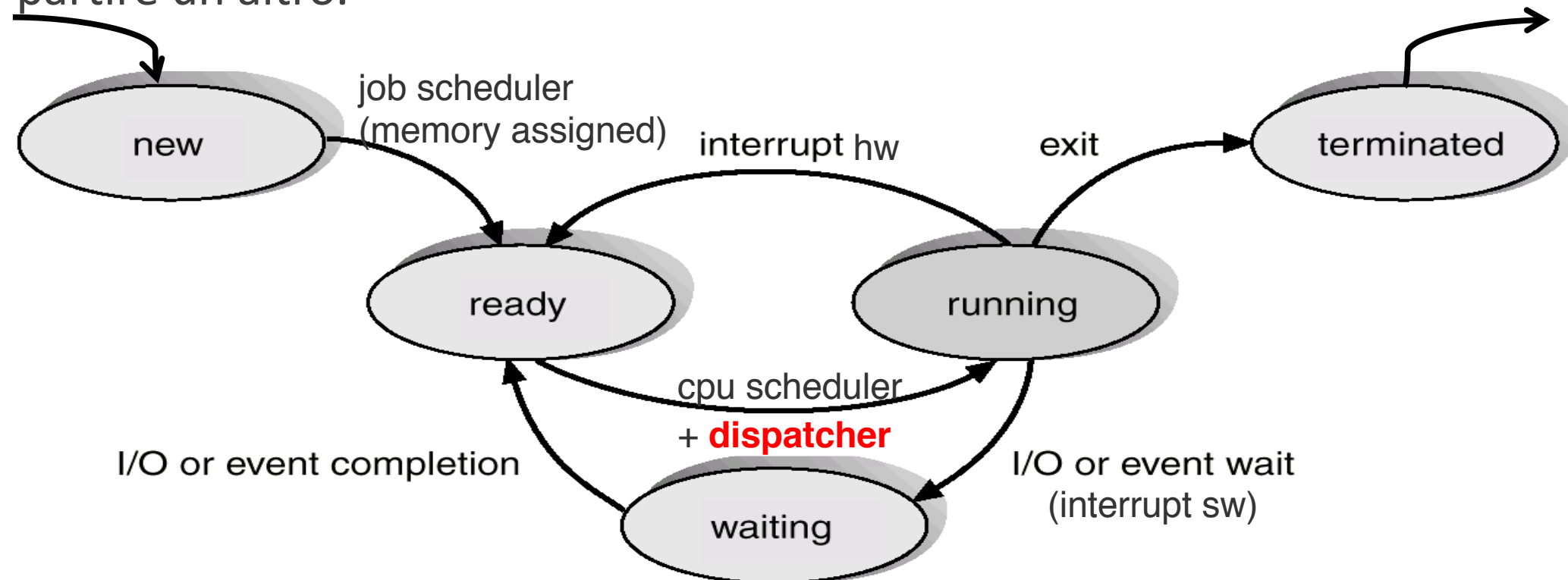
CPU Scheduler

- Seleziona un processo tra quelli presenti in memoria che sono pronti per l'esecuzione ed alloca la CPU a tale processo.
- Le decisioni riguardanti lo scheduling della CPU si possono prendere nelle seguenti circostanze:
 - 1. Un processo passa dallo stato di running allo stato di waiting.
 - 2. Un processo passa dallo stato di running allo stato di ready.
 - 3. Un processo passa dallo stato di waiting allo stato di ready.
 - 4. Un processo termina.



- Seleziona un processo tra quelli presenti in memoria che sono pronti per l'esecuzione ed alloca la CPU a tale processo.
- Le decisioni riguardanti lo scheduling della CPU si possono prendere nelle seguenti circostanze:
 - 1. Un processo passa dallo stato di running allo stato di waiting.
 - 2. Un processo passa dallo stato di running allo stato di ready.
 - 3. Un processo passa dallo stato di waiting allo stato di ready.
 - 4. Un processo termina.
- Scheduling senza diritto di prelazione (nonpreemptive) quando si assegna la CPU ad un processo, questo rimane in possesso della CPU fino a quando non la rilascia spontaneamente.
- Scheduling con diritto di prelazione (preemptive) quando il SO può obbligare un processo a rilasciare la CPU anche se il processo ne ha ancora bisogno.

- Dispatcher assegna il controllo della CPU al processo selezionato dal short-term scheduler; questo comporta:
 - Commutazione di contesto (switching context)
 - Passaggio da modalità kernel a modalità utente
 - Salto alla locazione di memoria appropriata per far ripartire il programma utente
- Latenza di Dispatch – il tempo necessario a fermare un processo e a far partire un altro.





CPU Scheduling

CPU SCHEDULING NEI SISTEMI MONOPROCESSORE

- ✓ Scheduling dei processi
 - CPU Scheduling nei sistemi monoprocessori
 - Criteri di scheduling
 - Algoritmi di scheduling
 - FCFS
 - SJF
 - Priorità
 - RR
 - Multilevel queue
 - CPU Scheduling nei sistemi multiprocessori
 - Lo scheduling in Linux

Criteri di Scheduling Monoprocessore

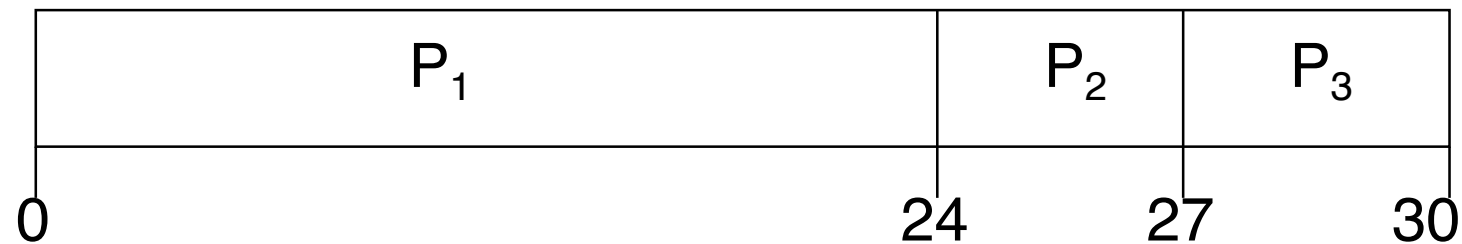
- **Utilizzo della CPU** – tenere la CPU occupata il più possibile
da **massimizzare**
- **Produttività** (Throughput) – numero di processi che completano la loro esecuzione per unità di tempo
da **massimizzare**
- **Tempo di Completamento** (Turnaround Time) – il tempo di esecuzione di uno specifico processo
da **minimizzare**
- **Tempo di attesa** – quanto tempo in totale un processo è rimasto nella coda di ready
da **minimizzare**
- **Tempo di risposta** – quanto tempo passa dal momento in cui la richiesta è stata sottomessa al momento in cui viene prodotta la prima risposta (non il suo output) – per i sistemi time-sharing
da **minimizzare**

First-Come, First-Served (FCFS)

Process Burst Time

P1	24
P2	3
P3	3

- Supponiamo che l'ordine di arrivo dei processi sia: P1 , P2 , P3
Il diagramma di Gantt risulta essere il seguente:

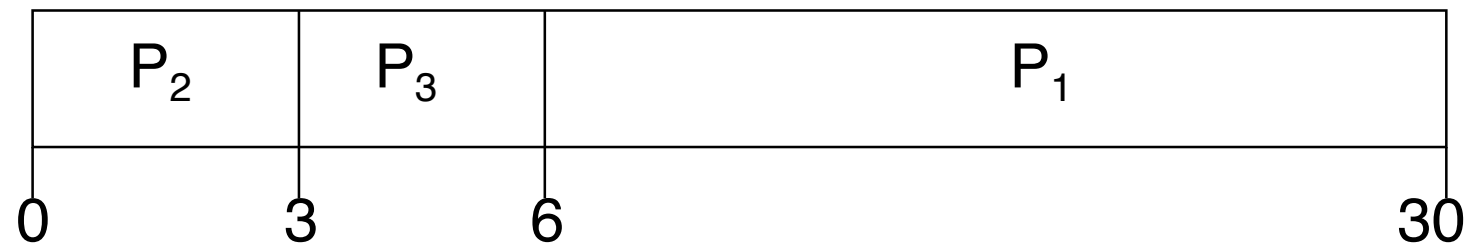


- Tempi di attesa: $P1 = 0; P2 = 24; P3 = 27$
- Tempo medio di attesa: $(0 + 24 + 27)/3 = 17$

- Supponiamo che l'ordine di arrivo sia invece il seguente:

P₂ , P₃ , P₁ .

- Il diagramma di Gantt diventa:



- Tempi di attesa: $P_1 = 6; P_2 = 0; P_3 = 3$
- Tempo medio di attesa: $(6 + 0 + 3)/3 = 3$
- Il miglioramento è considerevole.
- **Effetto convoglio:** I processi brevi si accodano ai processi lunghi.

Shortest-Job-First (SJF)

- Questo algoritmo associa a ogni processo la lunghezza del CPU burst successivo.
- La CPU viene assegnata al processo con il CPU burst successivo più breve.
- Due schemi:
 - Senza prelazione – il processo mantiene il possesso della CPU fino a quando non completa il CPU burst attuale.
 - Con prelazione – se arriva un nuovo processo con un CPU burst inferiore del CPU burst che resta al processo attualmente in esecuzione. Questo schema viene chiamato anche Shortest-Remaining-Time-First (SRTF).
- SJF è ottimale – minimizza il tempo medio di attesa.

Non-Preemptive SJF

Process	Arrival Time	Burst Time
---------	--------------	------------

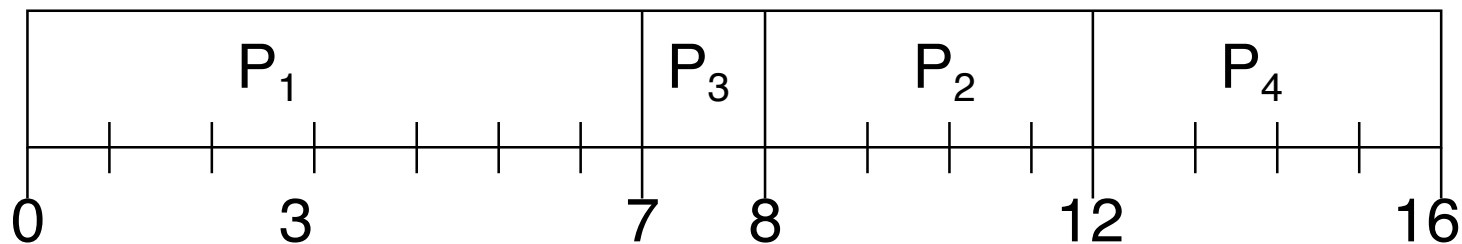
P1	0.0	7
----	-----	---

P2	2.0	4
----	-----	---

P3	4.0	1
----	-----	---

P4	5.0	4
----	-----	---

- SJF (non-preemptive)



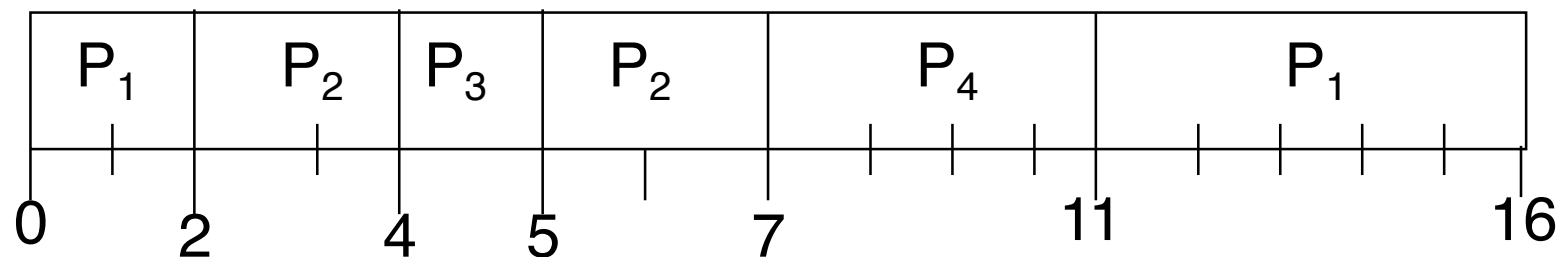
- Tempo medio di attesa = $(0 + 6 + 3 + 7)/4 = 4$

Preemptive SJF

Process Arrival Time Burst Time

P1	0.0	7
P2	2.0	4
P3	4.0	1
P4	5.0	4

■ SJF (preemptive)



■ Tempo medio di attesa =

$$\left(\underbrace{((0 - 0) + (11 - 2))}_{P1} + \underbrace{((2 - 2) + (5 - 4))}_{P2} + \underbrace{(4 - 4)}_{P3} + \underbrace{(7 - 5)}_{P4} \right) / 4 =$$

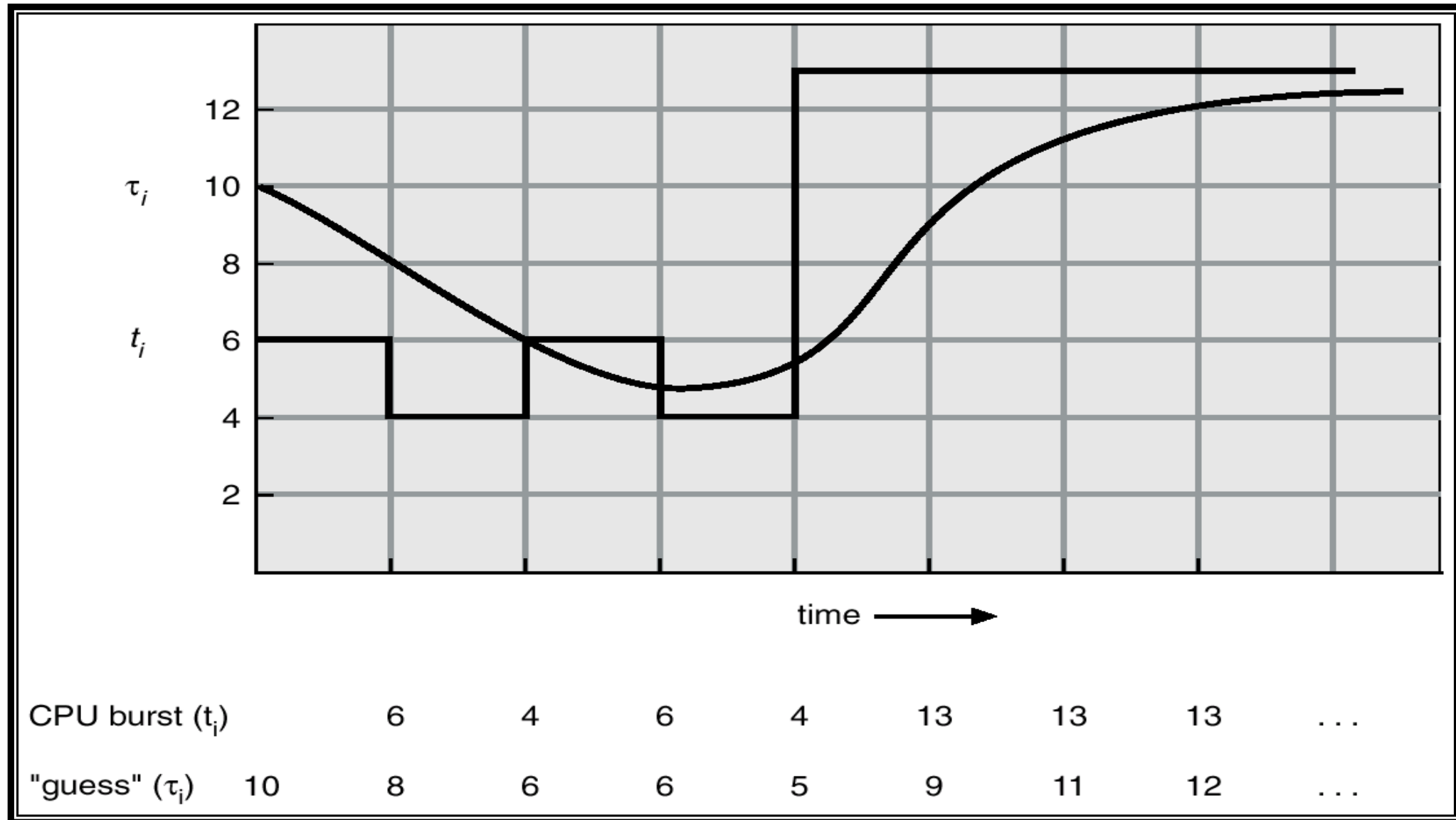
$$(9 + 1 + 0 + 2) / 4 = 3$$

Lunghezza del CPU Burst Successivo

- Può essere solo stimata.
- Si può utilizzare la media esponenziale dei CPU bursts precedenti.
 - t_n = lunghezza effettiva dell' n -esimo CPU burst
 - τ_n = lunghezza stimata dell' n -esimo CPU burst
 - α = peso, dove $0 \leq \alpha \leq 1$
 - c = stima della durata del primo CPU burst (condizione al contorno)

$$\begin{cases} \tau_{n+1} = \alpha t_n + (1 - \alpha) \tau_n \\ \tau_1 = c \end{cases}$$

Lunghezza del CPU Burst Successivo



- $\alpha = 0$
 - $\tau_{n+1} = \tau_n = \dots = \tau_1$
 - La storia recente del processo non conta.

- $\alpha = 1$
 - $\tau_{n+1} = t_n$
 - Conta solo l'ultimo CPU burst.

- Se espandiamo la formula otteniamo:

$$\begin{aligned}\tau_{n+1} &= \alpha \cdot t_n + (1 - \alpha) \cdot \alpha \cdot t_{n-1} + \dots \\ &\quad + (1 - \alpha)^j \cdot \alpha \cdot t_{n-j} + \dots \\ &\quad + (1 - \alpha)^n \cdot \tau_1 \\ &= \sum_{j=0}^{n-1} (1 - \alpha)^j \cdot \alpha \cdot t_{n-j} + (1 - \alpha)^n \cdot \tau_1\end{aligned}$$

- Siccome sia α sia $(1 - \alpha)$ sono minori o uguali ad uno, ogni termine ha un peso minore dei termini precedenti.

Scheduling per Priorità

- Ad ogni processo viene associata una priorità (un numero intero)
- La CPU viene allocata al processo con la priorità più alta (numero piccolo $\equiv$ alta priorità).
 - Con prelazione
 - Senza prelazione
- SJF è uno scheduling per priorità dove la priorità è la lunghezza del CPU burst successivo.
- Problema $\equiv$ **Starvation** – processi con bassi priorità potrebbero non ricevere mai la CPU.
- Soluzione $\equiv$ **Aging** – con il passare del tempo nella coda di ready, la priorità viene aumentata.

Round Robin (RR)

- Ogni processo riceve la CPU per una piccola unità di tempo (quanto di tempo), di solito 10-100 millisecondi. Allo scadere di questo quanto di tempo il processo è prelazionato ed aggiunto alla coda dei processi ready.
- La coda dei processi di ready viene gestita come una coda (FIFO) circolare.
- Se ci sono n processi nella coda di ready e il quanto di tempo è q , allora ogni processo non aspetta più di $(n-1)q$ unità di tempo.
- Le prestazioni dipendono da q
 - q molto grande $\Rightarrow$ RR diventa FCFS
 - q molto piccolo $\Rightarrow q$ deve essere molto più grande del tempo di commutazione di contesto, altrimenti l'overhead è troppo elevato.

Quanto di Tempo e Numero di Commutazioni

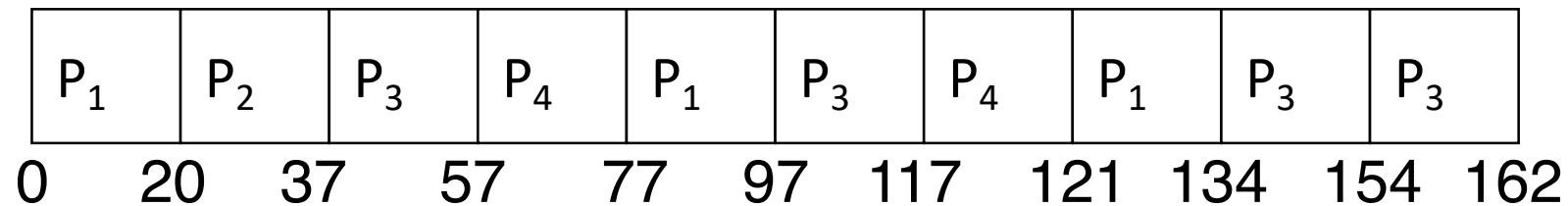
- Supponiamo di avere n processi con un CPU burst di durata pari a d .
- Supponiamo che il tempo di commutazione sia pari a D .
- Supponiamo che il quanto di tempo sia pari a q .
- La coda dei processi di ready viene gestita come una coda (FIFO) circolare.
- Se ci sono n processi nella coda di ready e il quanto di tempo è q , allora ogni processo non aspetta più di $(n-1)q$ unità di tempo.
- Le prestazioni dipendono da q
 - q molto grande $\Rightarrow$ RR diventa FCFS
 - q molto piccolo $\Rightarrow q$ deve essere molto più grande del tempo di commutazione di contesto, altrimenti l'overhead è troppo elevato.

RR con Quanto di Tempo = 20

Process Burst Time

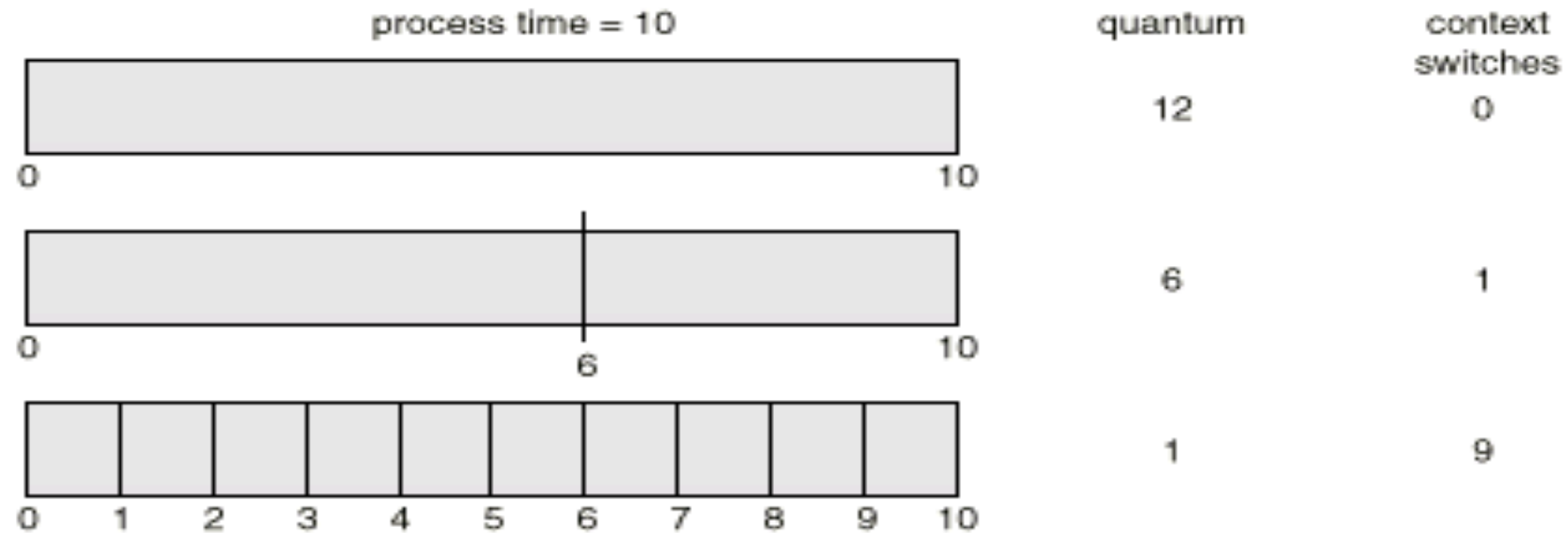
P1	53
P2	17
P3	68
P4	24

- Il diagramma di Gantt è il seguente:

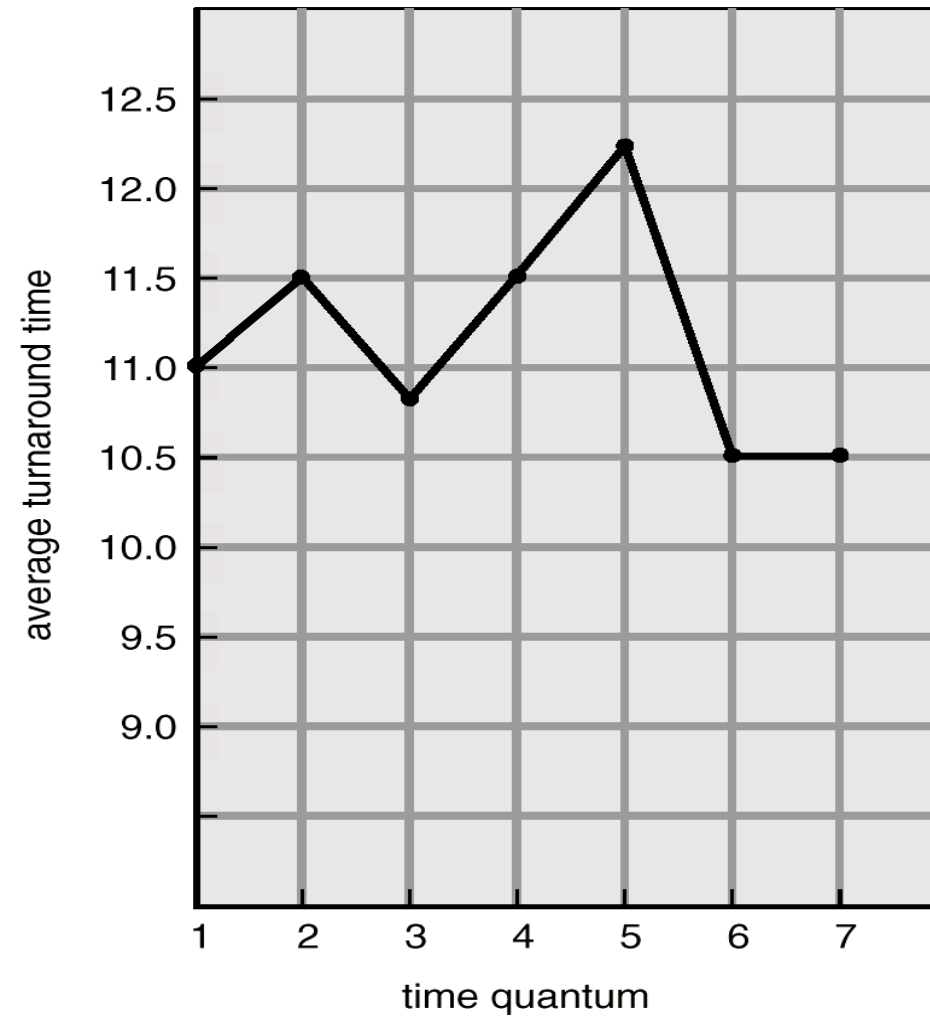


- Tipicamente ha un tempo medio di completamento maggiore di SJF, ma ha un tempo di risposta migliore.

Quanto di Tempo e Numero di Commutazioni



Tempo di Completamento e Quanto di Tempo



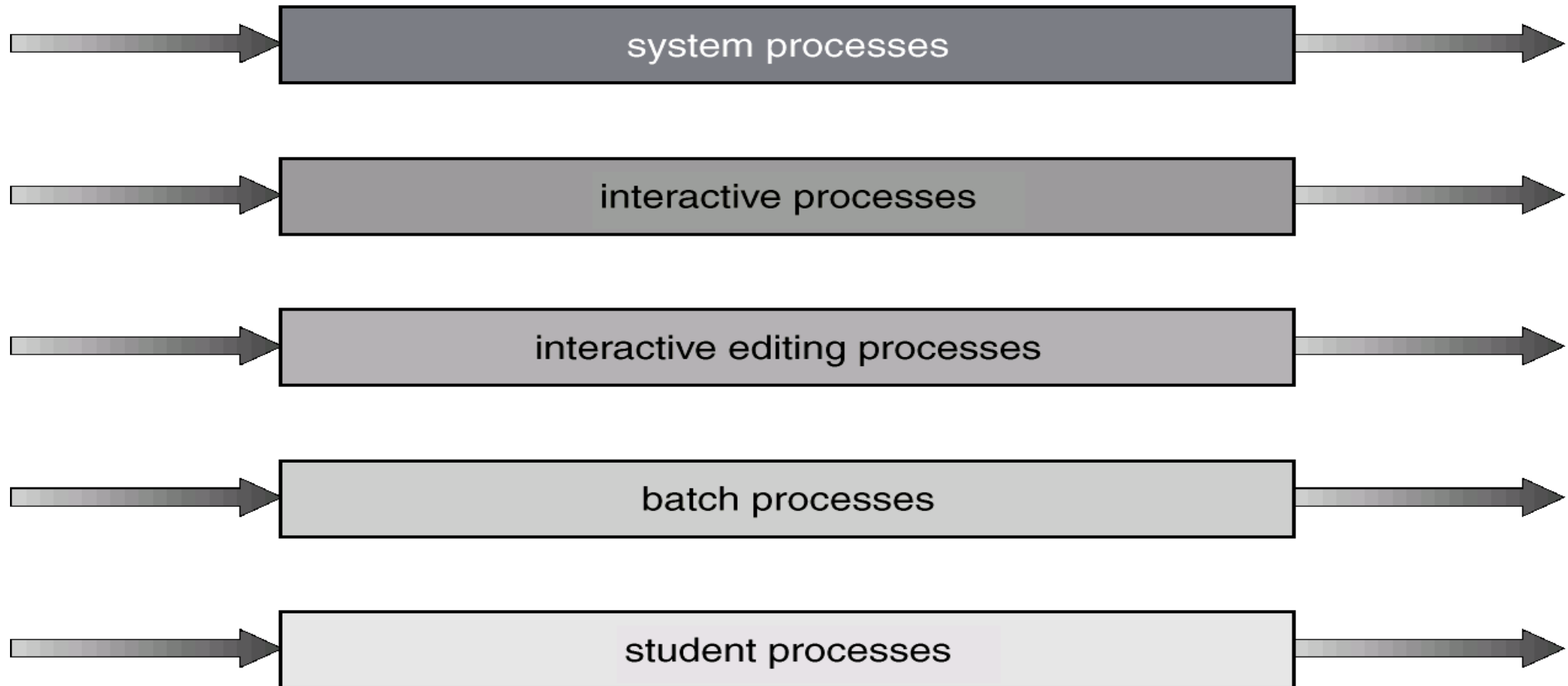
process	time
P_1	6
P_2	3
P_3	1
P_4	7

Multilevel Queue

- Questo algoritmo di scheduling suddivide la coda di ready in diverse code distinte. Ad esempio:
 - Processi foreground (interattivi)
 - Processi background (batch)
- Ogni **coda** adotta un proprio algoritmo di **scheduling**. Ad esempio:
 - foreground – RR
 - background – FCFS
- È inoltre necessario avere uno **scheduling tra le code**. Ad esempio:
 - Priorità fissa con prelazione; (per esempio vengono serviti prima tutti i processi foreground e poi quelli background). Possibilità di starvation.
 - Time slice – ogni coda ha a disposizione una certa quantità di tempo di CPU per i propri processi: ad esempio
 - 80% ai processi foreground in RR
 - 20% ai processi background in FCFS

Multilevel Queue

highest priority



lowest priority

Multilevel Feedback Queue

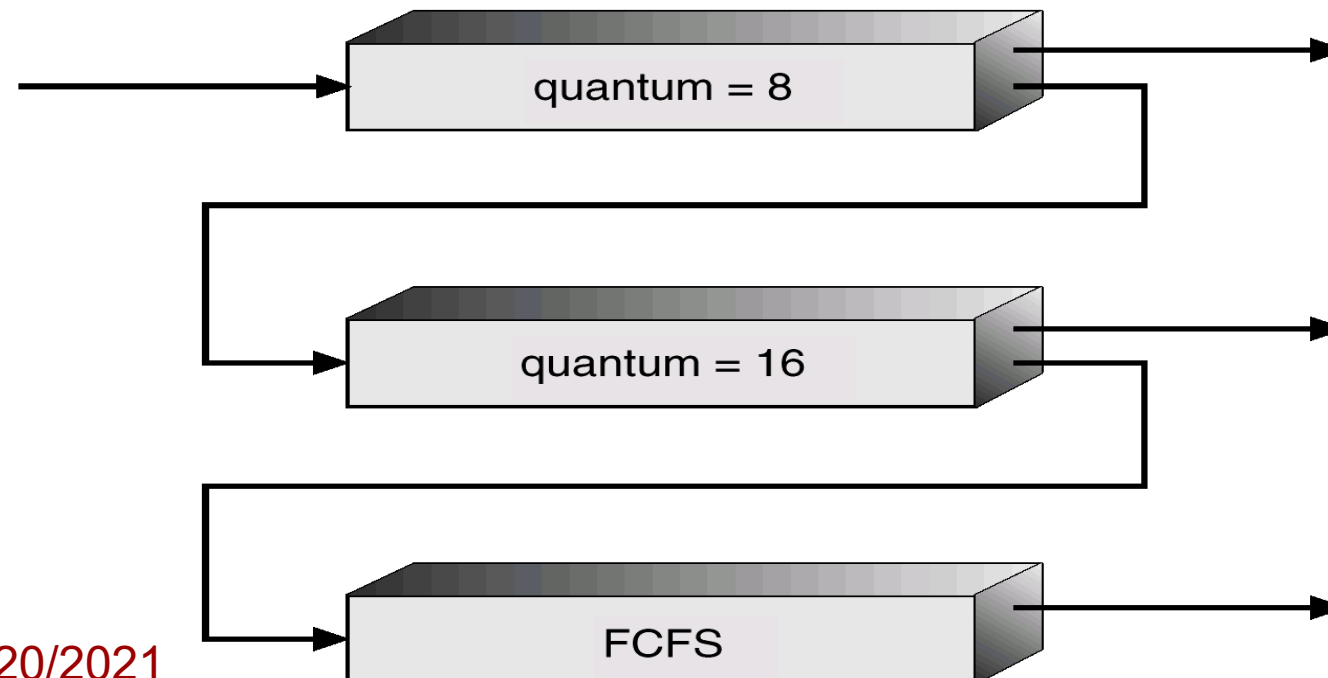
Un processo può passare da una coda ad un'altra.

- Ad esempio un processo che usa troppo la CPU si può spostare in una coda con priorità minore. Questo schema mantiene i processi con prevalenza di I/O e i processi interattivi nelle code con priorità elevata.
- Analogamente un processo che attende da troppo tempo si può spostare in una coda con priorità maggiore (ovvero questo è un modo per implementare l'aging).

Multilevel Feedback Queue

■ Scheduling

- Un nuovo processo entra nella coda Q0. Quando ottiene la CPU, riceve 8 millisecondi. Se non finisce in 8 millisecondi, il processo viene spostato nella coda Q1.
- Nella coda Q1, quando il processo ottiene di nuovo la CPU, riceve 16 millisecondi. Se ancora non bastano, al termine del quanto di tempo il processo è spostato in Q2.
- Nella coda Q2, quando il processo ottiene di nuovo la CPU, va avanti fino al suo completamento o a una sua sospensione.





CPU Scheduling

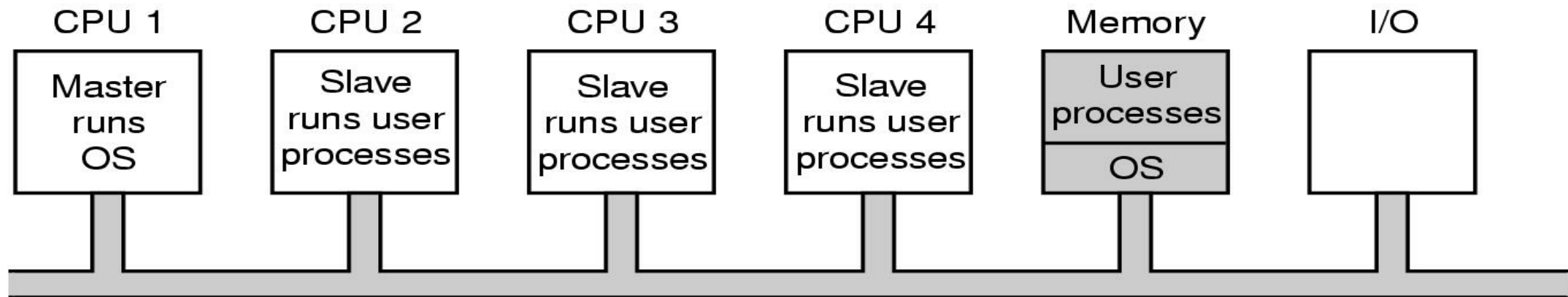
CPU SCHEDULING NEI SISTEMI MULTIPROCESSORE

-
- ✓ Scheduling dei processi
 - ✓ CPU Scheduling nei sistemi monoprocessori
 - CPU Scheduling nei sistemi multiprocessori
 - Introduzione e Criteri di scheduling
 - Algoritmi di scheduling: job-blind
 - Smart scheduling
 - Algoritmi di scheduling: job-aware
 - Gang scheduling
 - Dynamic partitioning
 - Lo scheduling in Linux

Asymmetric multiprocessing

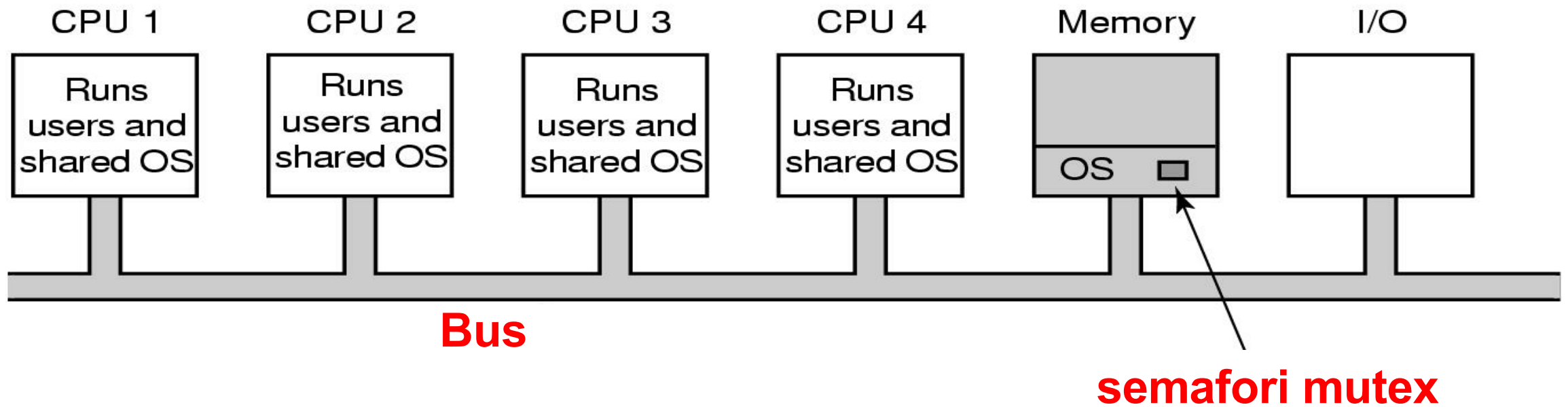
Solo un processore (Master) accede alle strutture dati di sistema

- Vantaggi: Si mantiene un buon bilanciamento del carico (tra le CPU Slave)
- Svantaggi: Ogni system call/interrupt deve essere diretto verso la CPU Master (perché è l'unica ad eseguire il codice del S.O. in questo schema) quindi se il numero di CPU é alto, il Master può diventare un collo di bottiglia.



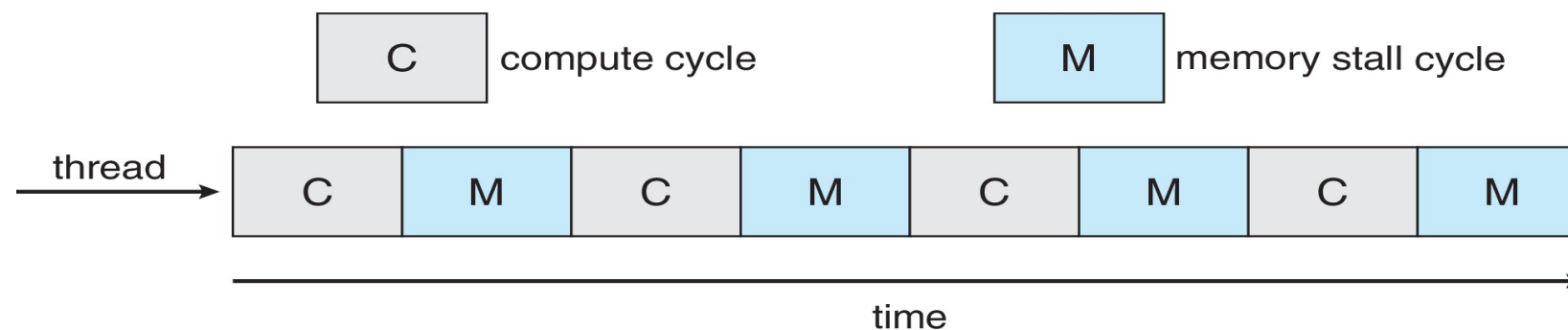
Symmetric multiprocessing (SMP)

- Ogni CPU può eseguire il codice del S.O.



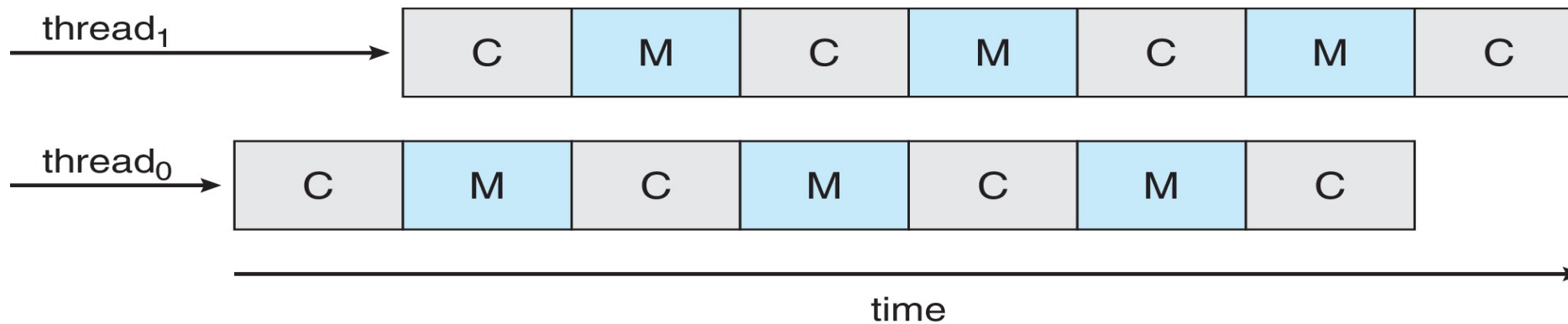
Multicore Processors

- Le architetture multicore permettono la presenza di thread multipli per ogni core.
- Un cpu-burst di un thread può essere a sua volta visto come un'alternanza di operazioni che coinvolgono la cpu (**compute cycle**) e operazioni che coinvolgono la memoria (**memory stall**)



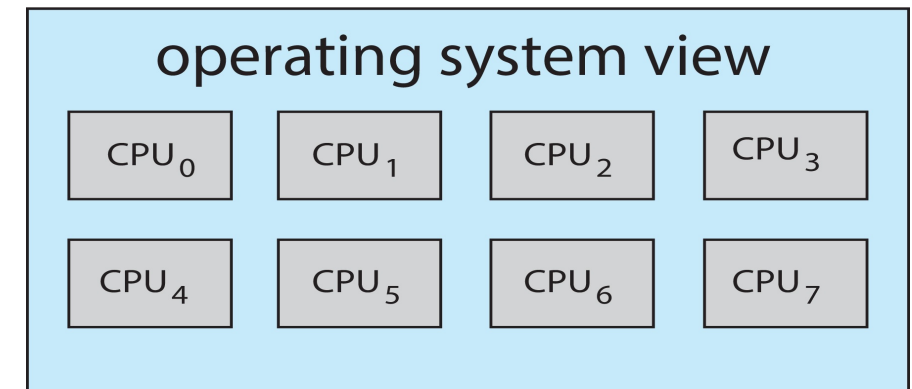
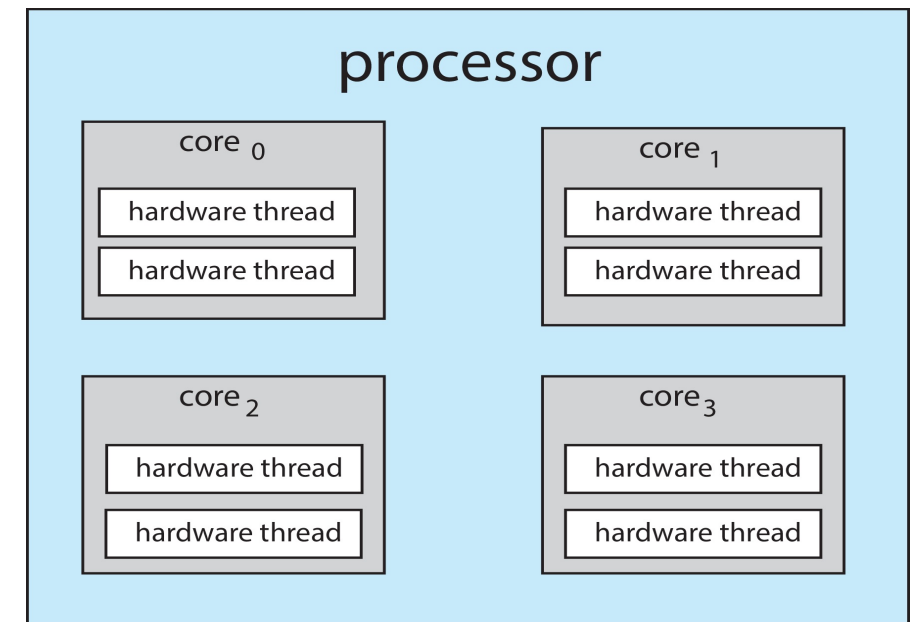
Multithreaded Multicore System

- Quando un thread è in memory stall, la cpu può essere ceduta ad un altro thread



Multithreaded Multicore System

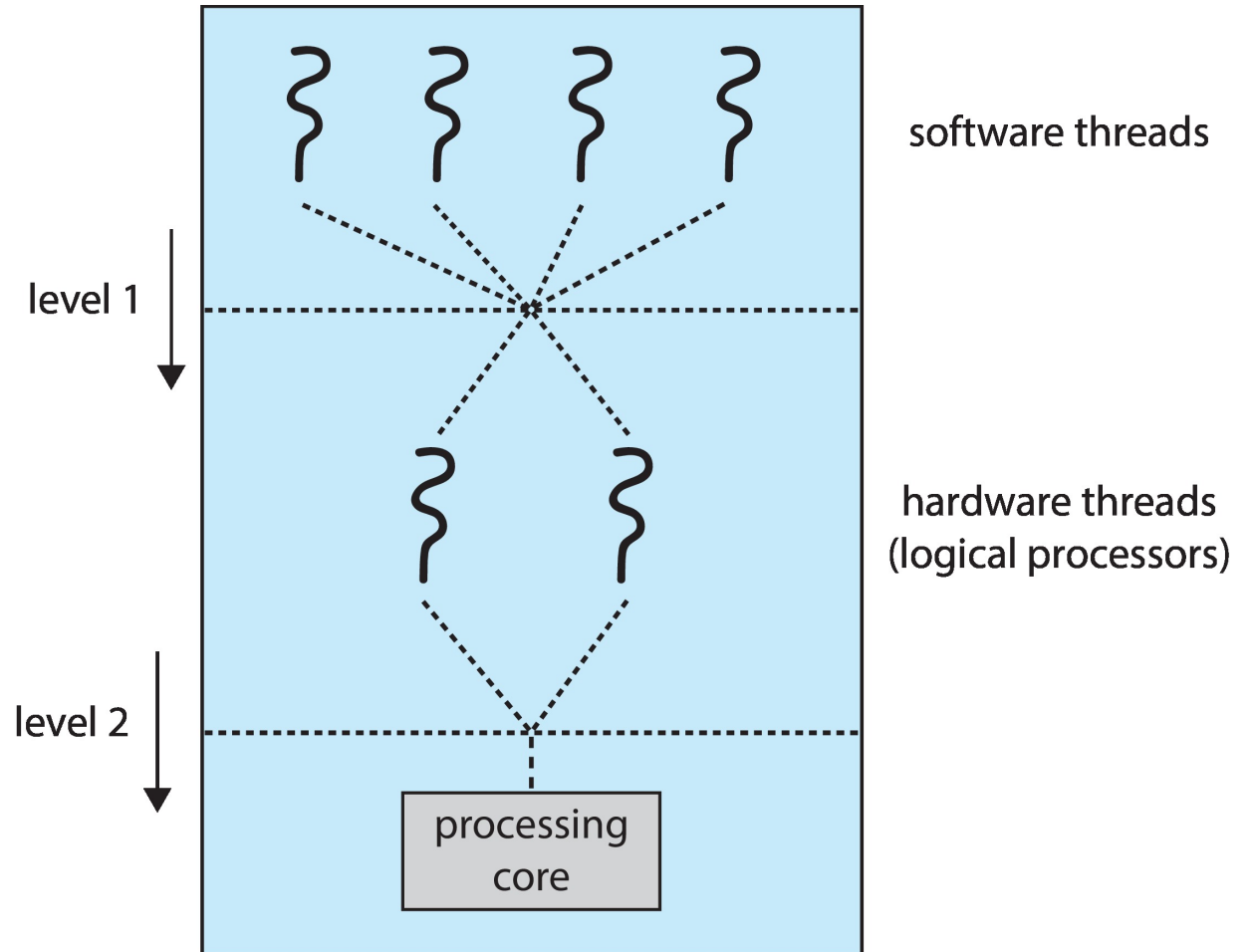
- Quando un thread è in memory stall, la cpu può essere ceduta ad un altro thread
- Assegnare hardware-thread multipli ad un unico core si chiama **chip-multithreading** (CMT)
 - Intel lo chiama **hyperthreading**.
 - Esempio: in un quad-core con 2 hardware threads per core, il S.O. vede 8 processori logici.



Multithreaded Multicore System

Due livelli di scheduling

1. Il S.O. decide quale software thread assegnare ad una cpu logica.
2. Ogni core decide quale hardware thread assegnare ad un dato core fisico (a cui fanno riferimento un insieme di cpu logiche)



Criteri di Scheduling Multiprocessore

Si passa da un problema a una dimensione (...quale fra i processi ready rendere running?) ad un problema bidimensionale (... e su quale CPU logica?)

Criteri:

- Gli obiettivi dello scheduling monoprocessore.
- **Parallelismo dei job**: massimizzare il parallelismo per sfruttare la concorrenza del job

da **massimizzare**

In alcuni casi per esempio non è utile minimizzare il tempo di esecuzione di un processo nel gruppo se non si minimizza anche quello di tutti gli altri (se il lavoro complessivo risulta terminato solo dopo la terminazione di tutti i processi nel gruppo).

- **Affinità con il processore**: relazione di un job con un particolare processore, la sua memoria locale (nei sistemi NUMA) e la sua cache

da **massimizzare**

In alcuni casi nella cache potrebbero esserci ancora suoi dati (e nel TLB potrebbero esserci ancora elementi della sua tabella delle pagine).

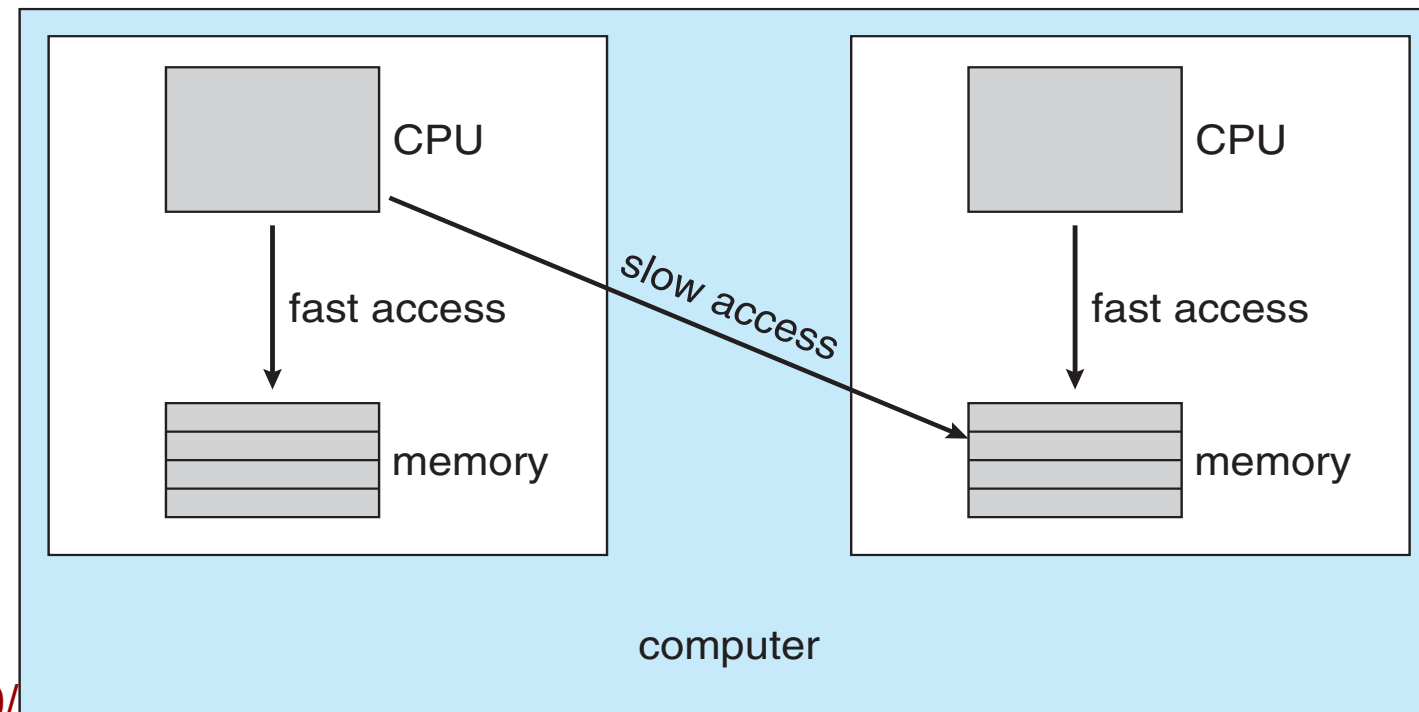
- **Bilanciamento del carico**: il carico di lavoro deve essere equamente distribuito tra i processori logici

da **massimizzare**

Non devono esserci thread in attesa quando ci sono processori inattivi

Affinità con il processore

- **Soft affinity** – il S.O. prova a mantenere i thread di uno stesso job nello stesso processore, ma non è garantito.
- **Hard affinity** – permette ad un job di indicare l'insieme dei processori dove devono girare i suoi thread.
- Utile per esempio nei sistemi NUMA (in questo caso si parla di S.O. **NUMA-aware**)



Bilanciamento del carico

Se il carico di lavoro è sbilanciato il S.O. può decidere di far migrare un certo numero di thread da un processore ad un altro per ristabilire l'equilibrio. Due possibili strategie:

- **Push migration** – si va alla ricerca di CPU sovraccariche, quando si trovano si fanno migrare dei thread verso altre CPU
- **Pull migration** – si va alla ricerca di CPU inattive, quando si trovano si fanno migrare dei thread verso queste CPU

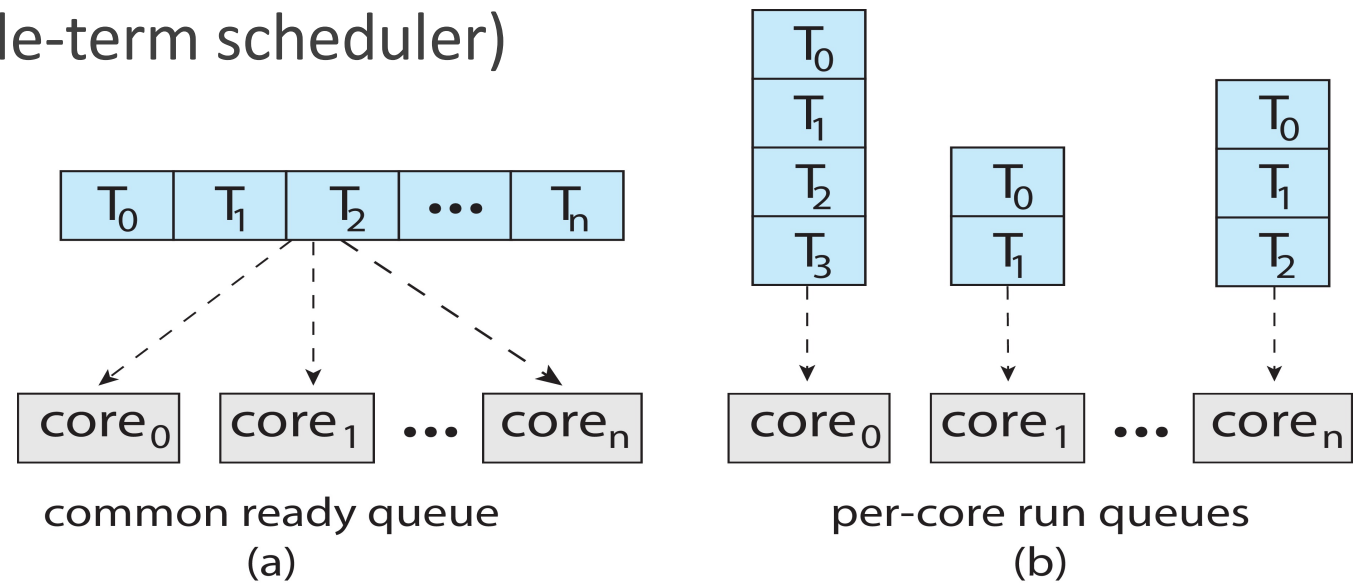
Classificazione degli algoritmi

- Una ready queue per processore
 - Algoritmi di scheduling monoprocessoore
 - Serve un algoritmo per il bilanciamento del carico (job scheduler e middle-term scheduler)

- Ready queue unica

- Job-blind
- Job-aware

- Il bilanciamento deve essere garantito dagli algoritmi di scheduling stessi



- Si possono estendere tutti gli algoritmi per sistemi monoprocesore.
- Esempi:
 - First-Come-First-Served (FCFS)
 - Round-Robin (RR)
 - Shortest Job First (SJF)
 - ...
- Semplici da realizzare e con un sovraccarico minimo
- Non tengono conto né del parallelismo né dell'affinità

- Ottimizzazione del RR
- Introdurre un flag che permetta di riconoscere un processo in sezione critica
- Quando un processo è in sezione critica e il quanto di tempo è scaduto, lasciare la CPU al processo fino al punto in cui rilascia la sua sezione critica.

Esercizio 1 - SJF

- Supponiamo di avere un sistema biprocessore e di avere nella coda di ready i seguenti processi:

Processo	Istante di arrivo	Burst Time
P1	0.0	10
P2	2.0	7
P3	4.0	4
P4	5.0	7
P5	6.0	6

- Calcolare il diagramma di GANTT applicando l'algoritmo SJF
- Calcolare il tempo medio di attesa

Esercizio 2 - RR

- Supponiamo di avere un sistema biprocessore e di avere nella coda di ready i seguenti processi:

Processo	Istante di arrivo	Burst Time
P1	0.0	10
P2	2.0	7
P3	4.0	4
P4	5.0	7
P5	6.0	6

- Calcolare il diagramma di GANTT applicando l'algoritmo RR con quanto di tempo pari a 5
- Calcolare il tempo medio di attesa

Esercizio 3 – Smart scheduling

- Supponiamo di avere un sistema biprocessore e di avere nella coda di ready i seguenti processi:

Processo	Istante di arrivo	Burst Time
P1	0.0	10
P2	2.0	7
P3	4.0	4
P4	5.0	7
P5	6.0	6

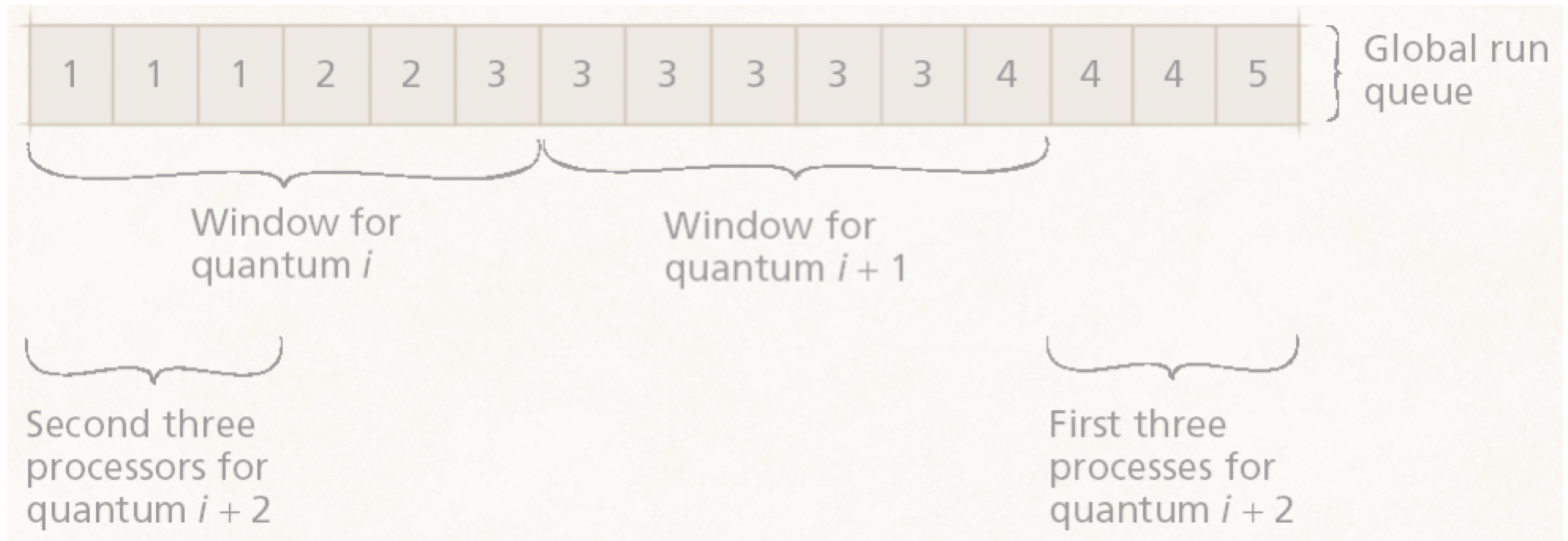
Tenendo presente che P1 dall'istante 2.0 all'istante 7.0 si trova all'interno di una sezione critica

- Calcolare il diagramma di GANTT applicando l'algoritmo smart scheduling con quanto di tempo pari a 5
- Calcolare il tempo medio di attesa

Coscheduling o Gang Scheduling

- Tutti i thread di un job sono posti in elementi adiacenti della ready queue, ogni job adiacente all'altro.
- Lo scheduler mantiene una finestra di dimensioni pari al numero di processori.
- I thread interni alla finestra sono eseguiti in parallelo per al massimo un quanto.
- Al termine del quanto la finestra si sposta al gruppo successivo.
- Cosa succede se un thread di una finestra passa a waiting?
Due possibili soluzioni:
 - il processore rimane inattivo fino alla fine del quanto di tempo;
 - la finestra viene estesa di un thread a destra.

Coscheduling o Gang Scheduling





Coscheduling o Gang Scheduling

- Utilizzo CPU **+++**
- Tempo di Attesa **+ -**
- Parallelismo dei Job **++**
- Affinità con il Processore **---**

Partizionamento dinamico

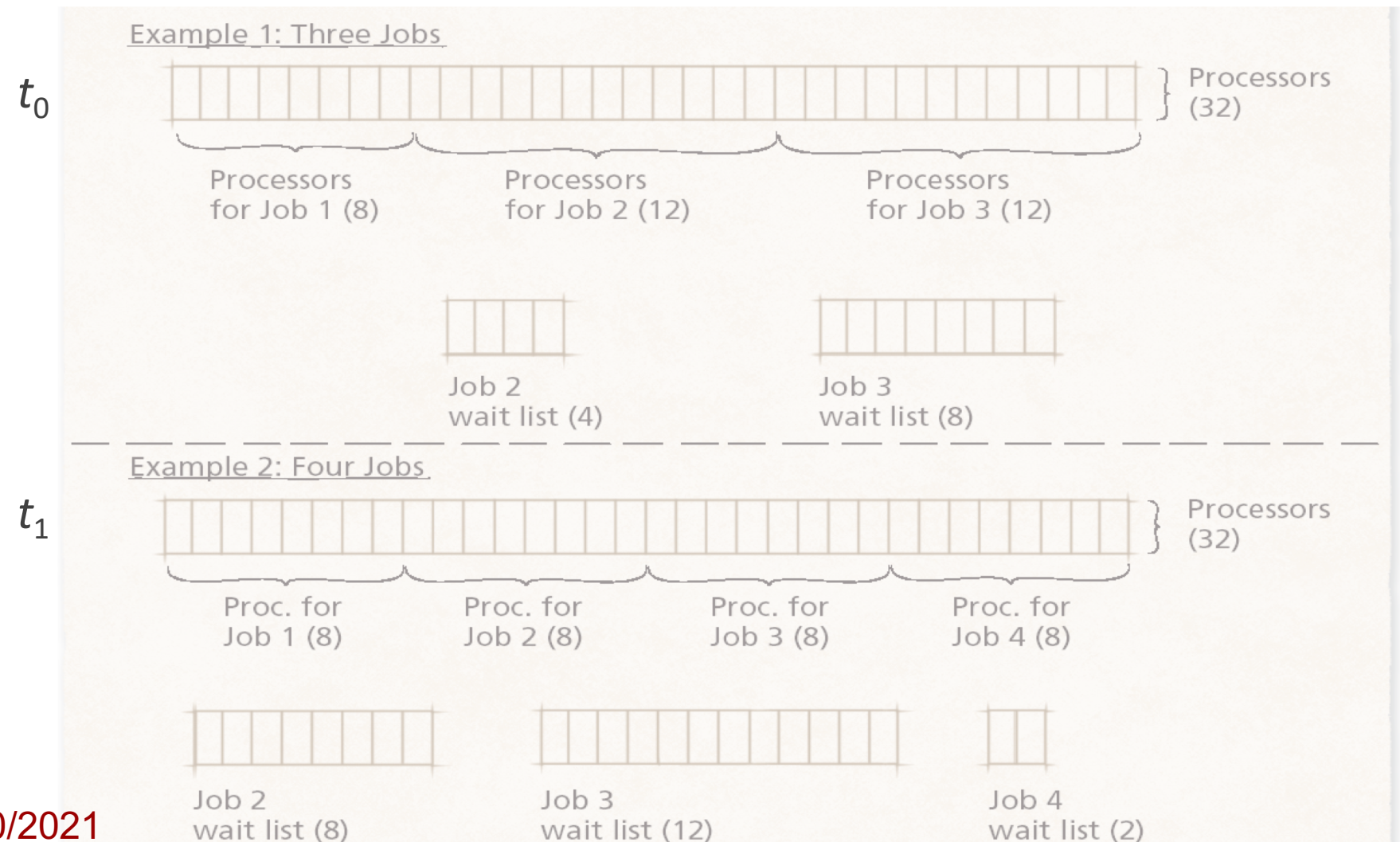
- Lo scheduler distribuisce equamente i job sui processori del sistema.
- Il numero di processori allocati ad un job è sempre minore o uguale al numero di thread eseguibili del job.
- Quando entra un nuovo job, il sistema aggiorna dinamicamente l'allocazione dei processori.

- Ottimizzazione uso CPU +++
- Ottimizzazione Tempo di Attesa +++
- Parallelismo dei Job ++
- Affinità con il Processore +++

Partizionamento dinamico

job 1 = 8 thread eseguibili job 2 = 16 thread eseguibili

job 3 = 20 thread eseguibili job 4 = 10 thread eseguibili



Esercizio 4 – Gang scheduling

- Supponiamo di avere un sistema con 4 processori e di avere nella coda di ready i seguenti processi multithread:

Processo	Thread	Burst Time
P1	T1.1	10
	T1.2	7
	T1.3	4
P2	T2.1	7
	T2.2	6
P3	T3.1	7
	T3.2	6
	T3.3	4

- Calcolare il diagramma di GANTT applicando l'algoritmo gang scheduling con quanto di tempo pari a 5

Esercizio 5 – Partizionamento dinamico

- Supponiamo di avere un sistema con 4 processori e di avere nella coda di ready i seguenti processi multithread:

Processo	Arrivo	Thread	Tempo Esec.
P1	0.0	T1.1	4
		T1.2	10
		T1.3	7
P2	0.0	T2.1	7
		T2.2	6
P3	5.0	T3.1	7
		T3.2	6
		T3.3	4

- Calcolare il diagramma di GANTT applicando l'algoritmo di partizionamento dinamico



CPU Scheduling

LO SCHEDULING IN LINUX



-
- ✓ Scheduling dei processi
 - ✓ CPU Scheduling nei sistemi monoprocessori
 - ✓ CPU Scheduling nei sistemi multiprocessori
 - Lo scheduling in Linux

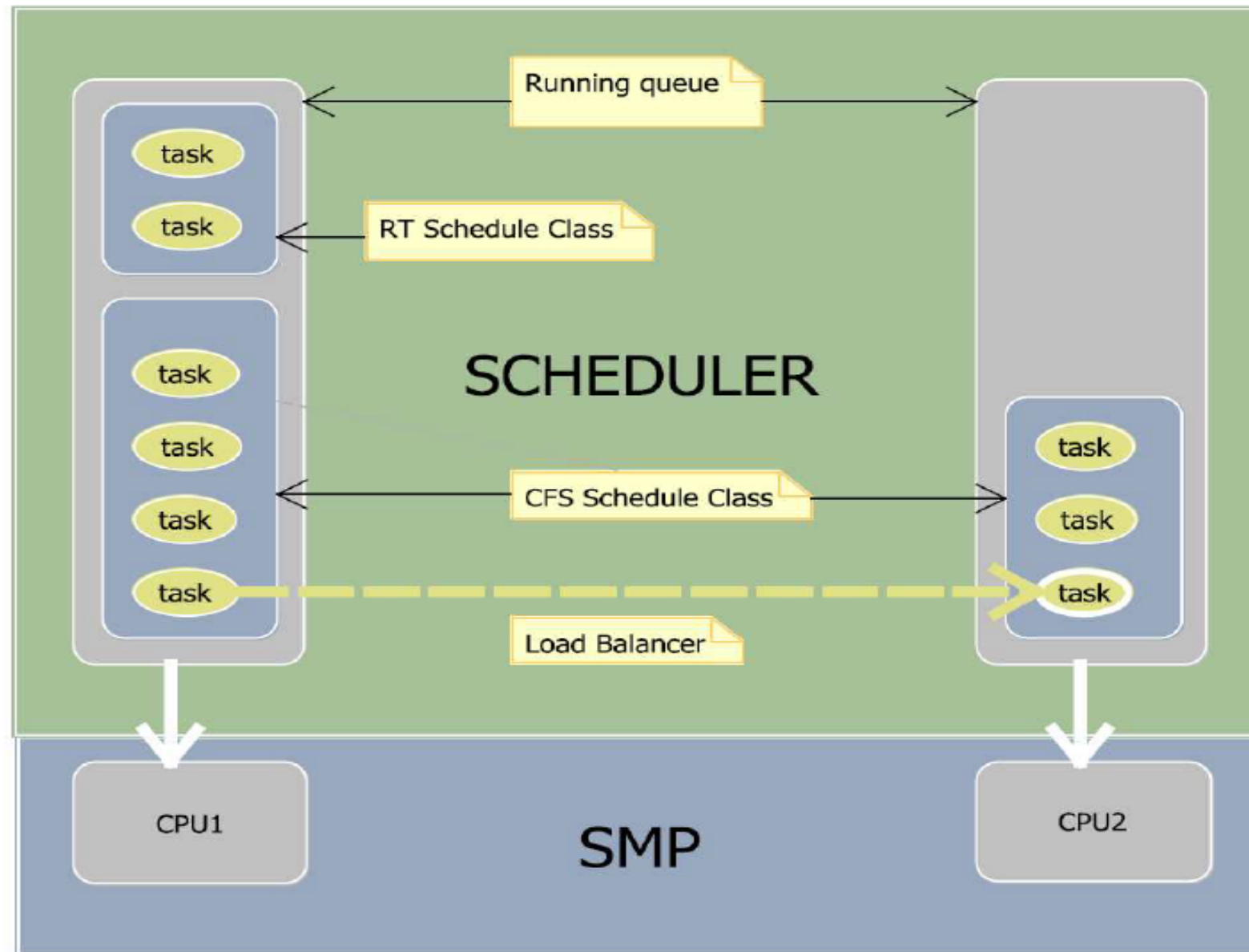
Diverse versioni di scheduler

- 2.2 – aggiunte le classi di scheduling & real-time policies
- 2.4 – real-time scheduler + UNIX-based scheduler
 - $O(n)$ scheduler
- 2.5 –
 - $O(1)$ scheduler
- 2.6.23 – Completely Fair Scheduler
 - $O(\log n)$ scheduler

Linux SMP Load Balancing

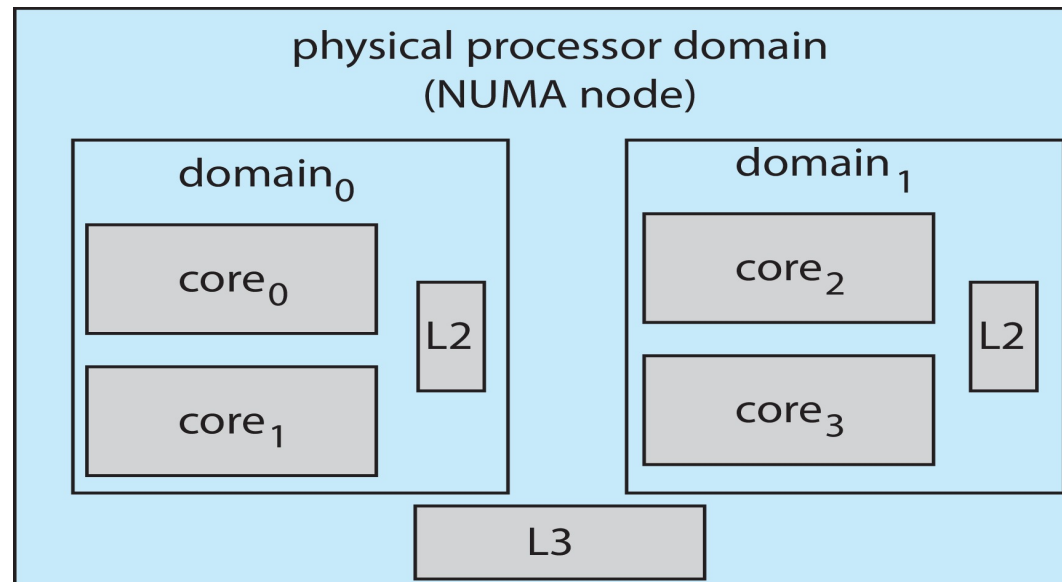
- Una coda per processore
 - Così quando un processore si libera, può selezionare un task senza bloccare la coda agli altri processori (visto che ognuno ha la sua)
- Periodicamente viene confrontata la lunghezza delle code sui differenti processori
 - I task vengono fatti migrare da un processore all'altro per bilanciare il carico di lavoro
 - Durante la migrazione, entrambe le code vengono bloccate (lock)

Scheduling in Linux



Linux Scheduling (Cont.)

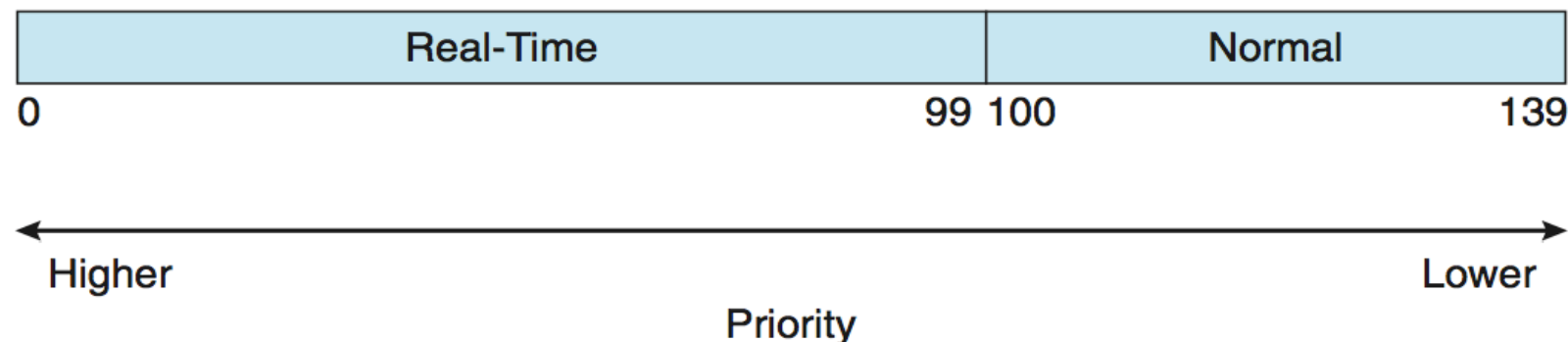
- Linux implementa il load balancing, ma è anche NUMA-aware.
- **Scheduling domain** è un insieme di CPU-core.
- I domain sono organizzati sulla base di quello che condividono (cioè memoria cache)
- L'obiettivo è impedire che i thread migrino da un domini all'altro.



Scheduling in Linux

Classi di Scheduling

- Linux implementa due classi di scheduling:
- `sched_fair`
 - È la classe di scheduling dei task in time-sharing
 - Priorità compresa tra 100 e 139
 - Utilizza l'algoritmo di scheduling Completely Fair Scheduler (CFS)
 - Vengono selezionati solo quando non ci sono task in classe `sched_rt`
- `sched_rt`
 - È la classe di scheduling dei task (soft-) real-time
 - Priorità compresa tra 0 e 99
 - Utilizza un algoritmo a code multiple



Scheduling in Linux

sched_rt: code multiple

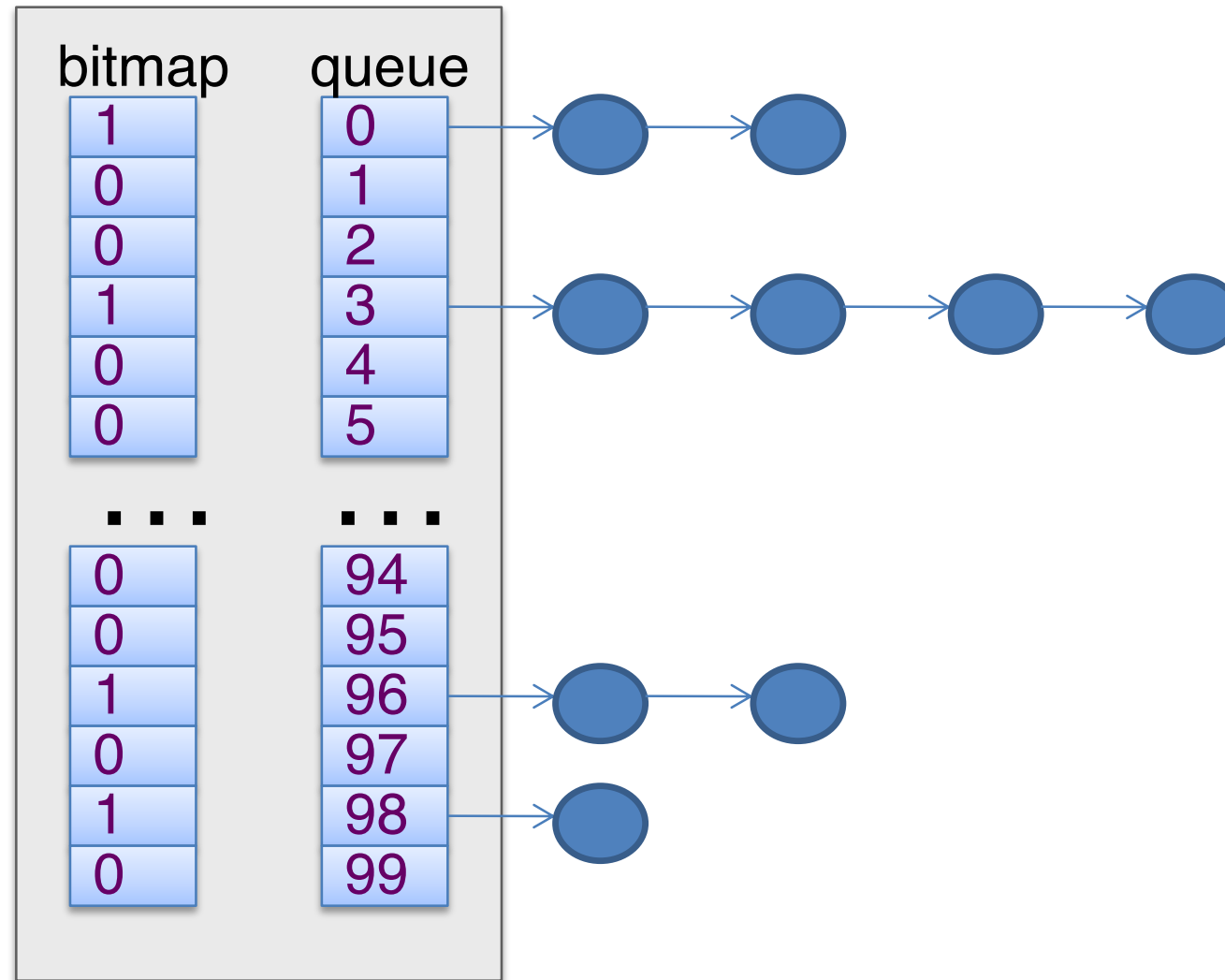
- Utilizza 100 code (una per priorità da 0 a 99)

<u>numeric priority</u>	<u>relative priority</u>		<u>time quantum</u>
0	highest		200 ms
•			
•			
•			
99			

Scheduling in Linux

sched_rt: code multiple

- Utilizza 100 code (una per priorità da 0 a 99)



L'algoritmo si basa sull'idea di dare il processore al task che ne ha maggiore necessità

- **Priorità**
 - priorità di default pari a 120
 - può essere modificata mediante il `nice`,
 - valore compreso tra -20 e +19,
 - quindi la priorità varia da 100 a 139
- **Fair scheduling**
 - la priorità viene usata come fattore di “accrescimento” del tempo di runtime (tempo passato in stato di running)
- **Quanto di tempo**
 - la priorità viene usata come decadimento del quanto di tempo quando il task è in running: più è bassa la priorità, più è alto il fattore di decadimento
- <https://gustavus.edu/+max/os-book/updates/CFS.html>

Prestazioni

- Utilizza un Red-Black Tree (RB-Tree) al posto di una coda
- Tempo
 - Tempo di **selezione** di task costante **$O(1)$**
 - gli alberi rosso-neri hanno un tempo di ricerca pari a $O(\log n)$ ma in Linux esiste un puntatore all'elemento che ha il vruntime minimo
 - Tempo di **inserimento** di un task pari a **$O(\log n)$**
 - Tempo di **cancellazione** di un task pari a **$O(\log n)$**

Quanto di tempo

- Target time – serve per controllare il tempo di risposta
 - $T = 20\text{ ms}$ – valore di default, adatto per una macchina desktop
 - $T = 100\text{ ms}$ – valore adatto per un server

- Peso – dipende dal valore di nice

- $w_i = 1024 \cdot (1.25)^{-\text{nice}_i}$

- Quanto di tempo

- $q_i = \frac{T \cdot w_i}{\sum_j w_j}$ j varia tra tutti i processi ready

- Esempio: $T=100\text{ms}$, $\text{nice}_1=0$, $\text{nice}_2=5$

- $w_1 = 1024$

- $w_2 = 335$

$$q_1 = \frac{100 \cdot 1024}{1024 + 335} = 75\text{ms} \quad q_2 = \frac{100 \cdot 335}{1024 + 335} = 25\text{ms}$$

Virtual runtime

- Viene utilizzato nell'RB-Tree per selezionare il task
- Runtime – il tempo di CPU speso dal task fino a quel momento
- Weighting factor
 - $f_i = 1024 / w_i = (1.25)^{+nice_i}$
- Virtual runtime
 - $vruntime_i = runtime_i \cdot f_i$
- Nice < 0 $\Rightarrow f < 1 \Rightarrow$ vruntime cresce più lentamente del tempo reale
- Nice = 0 $\Rightarrow f = 1 \Rightarrow$ vruntime cresce alla stessa velocità del tempo reale
- Nice > 0 $\Rightarrow f > 1 \Rightarrow$ vruntime cresce più velocemente del tempo reale

Un **RB-Tree** è un albero binario tale che:

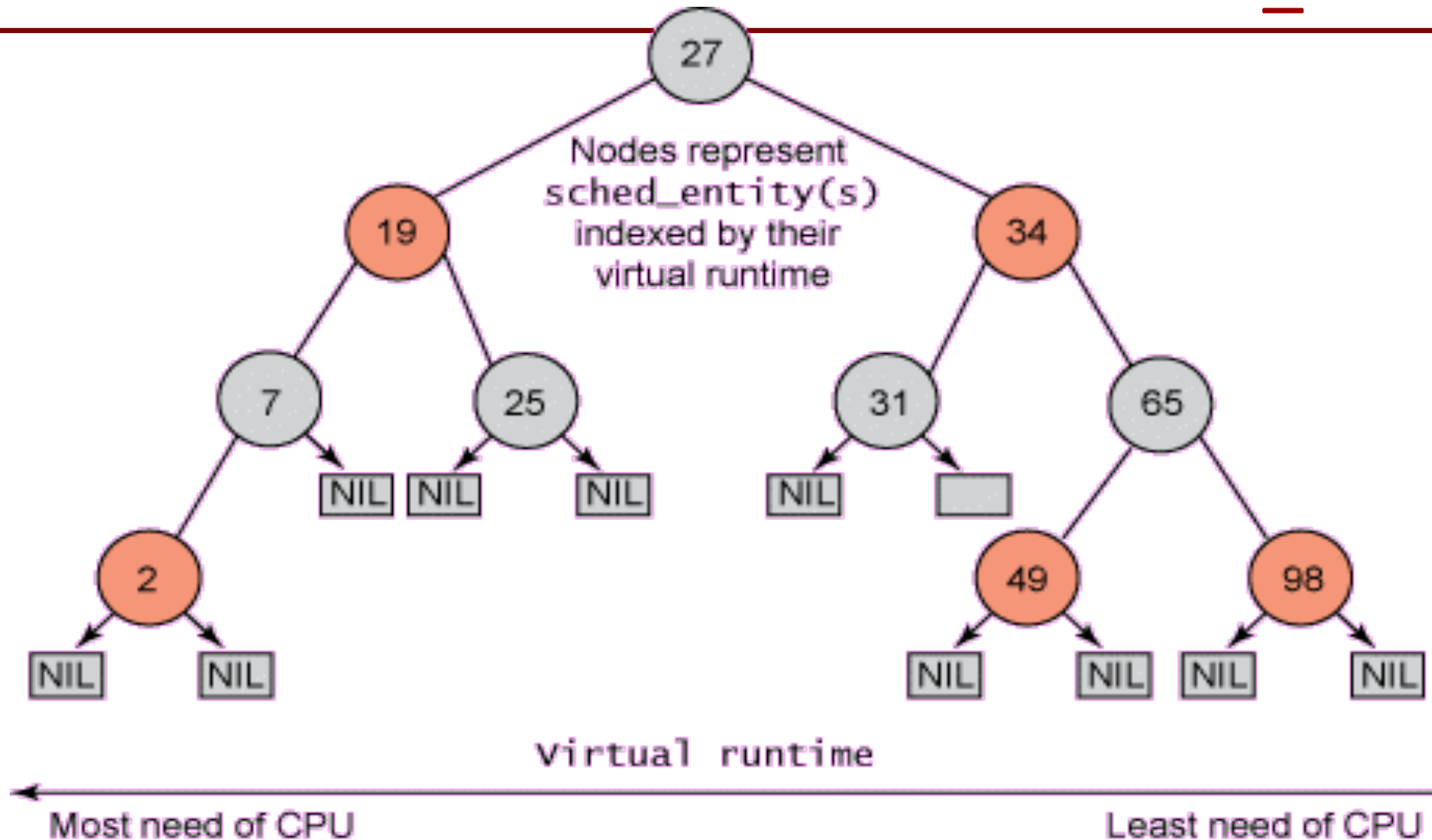
- Ogni nodo o è rosso o è nero
- Il nodo radice inizialmente è sempre nero
- Ogni nodo foglia è dello stesso colore della radice e ha valore NIL
- Entrambi i figli di un nodo rosso sono neri
- Ogni cammino da un nodo a una foglia nel suo sottoalbero contiene lo stesso numero di nodi neri

Il percorso radice-foglia più lungo ha una lunghezza pari al più al doppio della lunghezza del percorso più breve

- Inoltre, un RB-Tree è un albero binario tale che:
- Il sottoalbero di sinistra di un nodo v contiene nodi con valori minori di v o NIL
- Il sottoalbero di destra di un nodo v contiene nodi con valori maggiori o uguali di v o NIL
- Il valore di un nodo è il suo virtual runtime (vruntime)
- Lo scheduler seleziona sempre il nodo più a sinistra

Scheduling in Linux

sched_fair: CFS



Scheduling in Linux

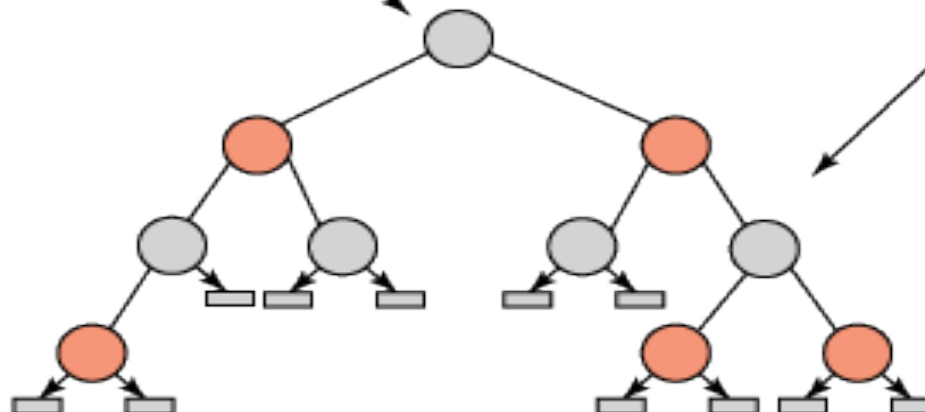
sched_fair: CFS

```
struct task_struct {
    volatile long state;
    void *stack;
    unsigned int flags;
    int prio, static_prio normal_prio;
    const struct sched_class *sched_class;
    struct sched_entity se;
    ...
};
```

```
struct ofs_rq {
    ...
    struct rb_root tasks_timeline;
    ...
};
```

```
struct sched_entity {
    struct load_weight load;
    struct rb_node run_node;
    struct list_head group_node;
    ...
};
```

```
struct rb_node {
    unsigned long rb_parent_color;
    struct rb_node *rb_right;
    struct rb_node *rb_left;
};
```



- Di conseguenza il task con il runtime più piccolo viene selezionato per primo (nodo più a sinistra dell'albero).
- La `rb_node` del task selezionato è rimossa dall'albero (lo stato del task è `running`).
- Quando lo stato del task ritorna `ready`, la `rb_node` viene reinserita nell'albero (probabilmente in una nuova posizione).
- Il nuovo nodo più a sinistra viene selezionato e la procedura si ripete.
- Se un task passa gran parte del tempo in `waiting` (interattivo), il suo `vruntime` è basso e quindi la sua priorità sarà più alta rispetto a quella di task che sono costantemente in `running` (batch).



Scheduling in Linux

Group Scheduling

- I task possono essere raggruppati in gruppi (la nozione di gruppo è equivalente a quella di job)
 - Esempio: task che genera altri task mediante `clone()`
- Tutti i task dello stesso gruppo hanno lo stesso runtime